

Chapter 1: Intro Integration

1.1 Merged intro content

1.2 The Hardest Explicit Construction

1.3 Strong vs. Weak Range Avoidance and the Linear Ordering Principle

TODO: I commented out stuff about top-down method and Kannan; we should make sure this goes in the intro, then bring back something quick to remind the reader of the context here

In recent breakthrough works of Chen, Hirahara, Ren and Li [1, 2], the first unconditional improvement to Kannan’s classical result¹. These works showed that the classes $S_2^P \subseteq ZPE^{NP} \subseteq \Sigma_2^E$ all contain a function of circuit complexity $2^n/n$, which is within a $(1 \pm o(1))$ factor of the maximum possible circuit complexity of a boolean function. The results are established by giving a new algorithm for Avoid which runs in the class FS_2^P , the functional variant of the decision class S_2^P .

The authors of [1, 2] made progress on a classical lower bound problem by discovering a new kind of algorithm for the total search problem Range Avoidance, a search problem in the class $TF\Sigma_2^P$. Since the introduction of $TF\Sigma_2^P$ in [5], there has been no follow up work developing a structural classification of the problems therein, despite the considerable attention that has been devoted to Range Avoidance and explicit construction problems more generally. The work of [1, 2] indicates that some problems in $TF\Sigma_2^P$ admit highly nontrivial algorithms, algorithms which have consequences in seemingly unrelated areas of complexity. Do such algorithms exist for all search problems in $TF\Sigma_2^P$? What more can we say about the relation of Avoid to other problems

¹This is the first improvement on finding the smallest complexity class with an *exponential* circuit lower bound. If the goal is to prove merely superpolynomial lower bounds, lower bounds for smaller classes can be established using Karp-Lipton theorems [3], an approach pioneered in Kannan’s original paper [4]. This method can at best show “sub-half-exponential” lower bounds, and only for infinitely many input lengths.

in this class? The purpose of this chapter is to address these two questions in particular, and more broadly to further the systematic study of $\text{TF}\Sigma_2^P$ beyond its introduction in [5], with an emphasis both on *inclusions* and *black box separations*. Prior to this work, no non-trivial separations between problems in $\text{TF}\Sigma_2^P$ were known.

In contrast to $\text{TF}\Sigma_2^P$, TFNP has received a thorough investigation over the past three decades and its structure is rather well understood in comparison. A major distinction we reveal between $\text{TF}\Sigma_2^P$ and TFNP is that the former has a plethora of *resource-constrained* subclasses, many of which can be separated from one another by explicit and natural search problems. By “resource-constrained subclass” we mean a class of search problems characterized by the existence of some resource-constrained algorithm which can find a solution. One example we will see is the class $\text{psFZPP}^{\text{NP}}$, the class of problems where for every input x , there is a canonical solution y_x which is output with high probability by some polynomial time NP-oracle algorithm. Another class is $\text{FP}^{\Sigma_2^{\text{NP}} \cap \Pi_2^P}$, consisting of those problems which can be solved in polynomial time with oracle access to a language in $\Sigma_2^P \cap \Pi_2^P$. We will introduce and study several other such subclasses, and exhibit four natural search problems which exhibit separations (in the decision tree model) between them. This situation is in stark contrast to the known structure of TFNP : while FNP does contain some intermediate resource-constrained classes, such as psFZPP and $\text{FP}^{\text{NP} \cap \text{coNP}}$ (these are roughly analogous to $\text{psFZPP}^{\text{NP}}$ and $\text{FP}^{\Sigma_2^{\text{NP}} \cap \Pi_2^P}$ mentioned above), it is known that in the decision tree model all of these classes collapse to FP . As a result, in the decision tree model the only resource-based distinctions amongst the standard TFNP problems is the distinction between FP and the rest of TFNP .

1.3.1 Overview of Main Results

A defining feature of search problems which distinguishes them from decision problems is that a given instance may have many different solutions. Indeed, if a search problem has a unique solution on every instance, then it may be equivalently phrased as a decision problem: given (x, i) , output the i^{th} bit of the unique solution for x . For many search problems of interest it is not clear

how to reduce them to a decision problem of the same complexity. For example given total search problem R we may define the decision problem LexFirst_R , where given an instance (x, i) we must output the i^{th} bit of the lexicographically first solution to x . While LexFirst_R is clearly at least as hard as R , in many cases it will be much harder and a reduction does not seem to exist in the opposite direction.

Say that $\mathcal{A}$ is a deterministic algorithm solving some search problem R . Then $\mathcal{A}$ naturally associates to each instance x a *canonical solution* $y_x := \mathcal{A}(x)$ which it outputs. In particular we can think of $\mathcal{A}$ as *defining a second search problem* R' with $(x, y) \in R'$ iff $\mathcal{A}(x) = y$; if $\mathcal{A}$ solves the original search problem R , then R is reducible to R' since $(x, y) \in R' \rightarrow (x, y) \in R$. Now, if the algorithm $\mathcal{A}$ lies in some restricted complexity class, then this places the same complexity upper bound derived search problem R' . The point of this is that when we have some nontrivial deterministic algorithm solving a search problem R , we may think of it as giving a reduction from R to a search problem with *unique solutions*.

For most of the classical problems in the class TFNP it is believed that there is *no decision problem* which captures their complexity precisely, a conjecture supported by black box separations [6]. More generally it is believed that for most of the standard problems $R \in \text{TFNP}$, any unique-solution problem R' which we can reduce R to must lie *outside of* TFNP². The new results in [2, 1] reveal that the situation for Range Avoid is different: because their Range Avoidance algorithms are deterministic and are upper bounded inside $\text{TF}\Sigma_2^{\text{P}}$, they imply that Range Avoidance is reducible to a problem in $\text{TF}\Sigma_2^{\text{P}}$ with unique solutions. To make this discussion more formal we need to define the class $\text{TF}\Sigma_2^{\text{P}}$ and its *unique solution* subclass $\text{TFU}\Sigma_2^{\text{P}}$:

Definition 1.1 ($\text{TF}\Sigma_2^{\text{P}}$ and $\text{TFU}\Sigma_2^{\text{P}}$). A polynomially-bounded³ search problem R lies in $\text{TF}\Sigma_2^{\text{P}}$ if it is a total search problem, and there exists a coNP verifier V so that $(x, y) \in R$ if and only if $V(x, y) = 1$. We say $R \in \text{TFU}\Sigma_2^{\text{P}}$ if moreover every instance has a unique solution.

In this terminology Li's result implies the following: Range Avoidance is polynomial-time

²This is equivalent to saying that we believe these standard problems, e.g. PPAD or PPP, not to lie in $\text{FP}^{\text{NP} \cap \text{coNP}}$.

³The “polynomially bounded” condition just means we are restricting attention to search problems whose solutions have polynomially bounded length.

reducible to a problem in $\text{TFU}\Sigma_2^{\text{P}}$. His result is in fact a significant strengthening of this, but for now we focus on this particular consequence, which seems quite unintuitive on the surface: given an instance $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ of Range Avoidance, there are at least 2^n distinct solutions, and it is not clear how to narrow down to any particular solution which is “more special” than the others. Our first contribution is to clarify this result in the following way: we introduce a *natural* $\text{TF}\Sigma_2^{\text{P}}$ search problem, whose containment in $\text{TFU}\Sigma_2^{\text{P}}$ is obvious from the definition, and then show that Range Avoidance reduces to this problem:

Definition 1.2 (Linear Ordering Principle (LOP)). Given $\prec: \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ specified by a boolean circuit, find a witness that $\prec$ does not define a total ordering on $\{0, 1\}^n$, or else find the minimal element in the ordering it defines. A witness that $\prec$ does not define a total order consist of $x, y, z \in \{0, 1\}^n$ such that one of the following holds: (a) $x \prec x$; (b) $x \neq y$, $x \not\prec y$, and $y \not\prec x$; (c) $x \prec y \prec z$ and $x \not\prec z$.

This problem was originally defined in the context of Bounded Arithmetic in [7]. While this problem does not literally have unique solutions as stated, it has a property which we show is morally equivalent. Observe that one type of solution in this problem is easier to verify than the other: a witness that $\prec$ fails to define a linear order can be verified in polynomial time, whereas a candidate minimum element can only be verified in coNP. Moreover, if an instance has no easily verifiable solutions, then it has a unique solution. This follows from the fact that every linear order has a unique minimal element. We summarize this by saying that LOP has *essentially unique solutions*; a very easy argument (Section ??, Lemma ??) shows that any problem with essentially unique solutions is reducible to a problem in $\text{TFU}\Sigma_2^{\text{P}}$. We then prove:

Theorem 1.3. *Range Avoidance is polynomial time reducible to LOP.*

We believe this result goes a long way in explaining the new upper bounds for Range Avoidance. In particular our reduction isolates the two key steps in Li’s algorithm which allow us to single out a special *low complexity canonical solution* for Range Avoidance: the first step prepares a special subset of solutions using a certain tree-like iteration construction, and the second step

singles out a fixed canonical solution among these by defining a certain *total ordering* on these special solutions. Recall our comment under Definition ?? that Li’s result is in fact stronger than a reduction from Range Avoidance to a problem in $\text{TFU}\Sigma_2^P$: more generally he shows that Range Avoidance lies in the complexity class FS_2^P (we will review the definition of this class in Section ??). Our result subsumes this upper bound as well, since another very direct argument shows that LOP lies in FS_2^P . Thus our result gives the current best upper bound on the Range Avoidance problem, and hence on the Kannan’s classical problem of constructing the truth table of a hard boolean function. **TODO: remove this Kannan comment? And talk about the followup work by Volkavic**

At this point we have seen that there are search problems in $\text{TF}\Sigma_2^P$ which seem on the surface to have no distinguished solutions, but which nonetheless can be reduced to problems in $\text{TFU}\Sigma_2^P$ by some highly non-obvious means. This naturally points to the following question: are all problems in $\text{TF}\Sigma_2^P$ reducible to $\text{TFU}\Sigma_2^P$? We give a negative answer in the decision tree model. Our separation is exhibited by the following relative of Range Avoidance which was introduced originally in [5] under the name *Empty*:

Definition 1.4 (Strong Avoid). Given $f : \{0, 1\}^n \setminus \{0^n\} \rightarrow \{0, 1\}^n$, find $y \in \{0, 1\}^n \setminus \text{range}(f)$.

From now on we will refer to Range Avoidance as “Weak Avoid” to distinguish it from Strong Avoid. Our black box separation will be significantly stronger than just showing that Strong Avoid is not reducible to a problem in $\text{TFU}\Sigma_2^P$; we will show more generally that Strong Avoid cannot be solved by making non-adaptive queries to any language in Σ_2^P , which is equivalent to proving size lower bounds for depth-3 AC_0 circuits solving Strong Avoid:

Theorem 1.5. *In the decision tree model, Strong Avoid is not in $\text{FP}_{||}^{\Sigma_2^P}$. More specifically, let $C : \{0, 1\}^{(N-1)\log N} \rightarrow \{0, 1\}^{N^\epsilon}$ be a depth-3 circuit of size 2^{N^ϵ} and let $D : \{0, 1\}^{N^\epsilon} \rightarrow \{0, 1\}^{\log N}$ be an arbitrary postprocessing function, where $N = 2^n = |\{0, 1\}^n|$. Then provided ϵ is sufficiently small, $D \circ C$ cannot solve Strong Avoid: there must be some input $f : [N - 1] \rightarrow [N]$ so that $D(C(f))$ fails to find a $y \notin \text{range}(f)$.*

This immediately implies non-reducibility to $\text{TFU}\Sigma_2^P$, since any problem $R \in \text{TFU}\Sigma_2^P$ with

unique solutions can be solved with non-adaptive queries to the language $\{(x, i) \mid (x, y) \in R \rightarrow y_i = 1\}$ which lies in Σ_2^P . The connection of these kinds of separations to depth 3 AC^0 lower bounds was spelled out in a recent paper of Vyas and Williams [8] for the case of *Weak Avoid*: their work established that nontrivial upper bounds for Weak Avoid are equivalent to certain depth 3 circuits solving the so-called “Missing String” problem: given an explicit list of 2^{n-1} n -bit strings, output a string not in the list. The input size here is $\approx N = 2^n$, and the question is whether a depth 3 circuit exists of size polynomial or quasipolynomial in N . Note that the Missing String problem is simply the black-box variant of Weak Avoid. Li’s result showed that quasipolynomial size depth-3 circuits for this problem actually *do exist*, solving the original question of Vyas and Williams in the positive. We show that if the problem is modified so that the list of strings has length $N - 1$ rather than $\frac{N}{2}$, then depth-3 circuits require exponential size to solve this problem. We note that our lower bound holds against a stronger class of circuits than what was originally considered by Williams and Vyas: in their model the depth 3 circuit is of the form $C : \{0, 1\}^{(N-1) \log N} \rightarrow \{0, 1\}^{\log N}$ and must output the exact solution to Avoid. Here we allow C to output an arbitrary string in $\{0, 1\}^{N^\epsilon}$, which can then be postprocessed arbitrarily to construct a solution to Avoid.

By a simple reduction, we also obtain quasipolynomial depth-3 circuit size lower bounds even for *moderately weak* Avoid instances with domain $[N]$ and codomain $[N + N/\log^{O(1)} N]$. This result gives a complete characterization of the degree of “Weakness” necessary to obtain quasipolynomial size depth 3 circuits: if the codomain has size $N + N/\log^{O(1)} N$ then circuits of quasipolynomial size suffice, and if its size is $N + N/\log^{\omega(1)} N$ then they do not; see Lemma ?? for details.

Our last main result exhibits a more fine-grained separation amongst the subclasses of $TF\Sigma_2^P$. Above we have highlighted one important distinction, between the problems which are reducible to $TFU\Sigma_2^P$ and those which aren’t. However both of the problems we’ve seen so far which reduce to $TFU\Sigma_2^P$ also have an additional property: they are solvable by a polynomial time randomized algorithm using a NP oracle. For Weak Avoid this follows from its definition, while for LOP it follows from its containment in the class FS_2^P . We show that this is not possible for all problems reducible to $TFU\Sigma_2^P$. Our separation is exhibited by the following natural search problem:

Definition 1.6 (Strong 1-1 Avoid). Given $f : \{0, 1\}^n \setminus \{0^n\} \rightarrow \{0, 1\}^n$, find a pair $x \neq y$ in $\{0, 1\}^n \setminus \{0^n\}$ such that $f(x) = f(y)$, or else $y \in \{0, 1\}^n \setminus \text{range}(f)$,

Observe that Strong 1-1 Avoid enjoys the same property as LOP of having *essentially unique solutions*: it is easy to verify the collision solutions $f(x) = f(y) \wedge x \neq y$, and any instance with no collision solutions has a unique solution. This follows from the fact that any injective function $f : [N] \rightarrow [N+1]$ misses exactly one point in its codomain. Hence, like Weak Avoid and LOP, the problem Strong 1-1 Avoid is reducible to $\text{TF}\Sigma_2^P$. However we prove the following lower bound:

Theorem 1.7. *In the decision tree model, Strong 1-1 Avoid is not solvable in FBPP^{NP} .*

Aside from revealing further the structure of $\text{TF}\Sigma_2^P$, this result yields a new separation for *decision classes* which was not previously known:

Theorem 1.8. *In the decision tree model, $\Sigma_2^P \cap \Pi_2^P \not\subseteq \text{BPP}^{\text{NP}}$. In particular $\Sigma_2^P \cap \Pi_2^P \not\subseteq S_2^P$.*

In the next two subsections we will describe these main results in some more technical detail. A diagram of the structure of $\text{TF}\Sigma_2^P$ and our main results is given in Figure ??; some classes in this diagram will not be defined until Section ??.

1.3.2 AC^0 Lower Bounds and Class Separations

Our two main separations show that Strong Avoid has no non-trivial upper bound inside of $\text{TF}\Sigma_2^P$ (Theorem ??), and that Strong 1-1 Avoid has no randomized NP-oracle algorithm (Theorem ??). The first lower bound is the more involved of the two, and requires proving a novel depth 3 AC^0 circuit lower bound for a total search problem, which appears to be the first circuit lower bound of this kind.

Theorem ?? yields a very fine-grained separation for the Strong Avoid problem. It is easy to construct a depth 4 circuit $C : \{0, 1\}^{(N-1)\lceil \log N \rceil} \rightarrow \{0, 1\}^{\lceil \log N \rceil}$ solving Strong Avoid, where moreover the bottom fan-in is only $O(\log N)$ [8]. More strongly, it is possible to construct a depth $O(\log N)$ decision tree, which at each step queries a depth 3, $\text{poly}(N)$ size circuit on the input f ,

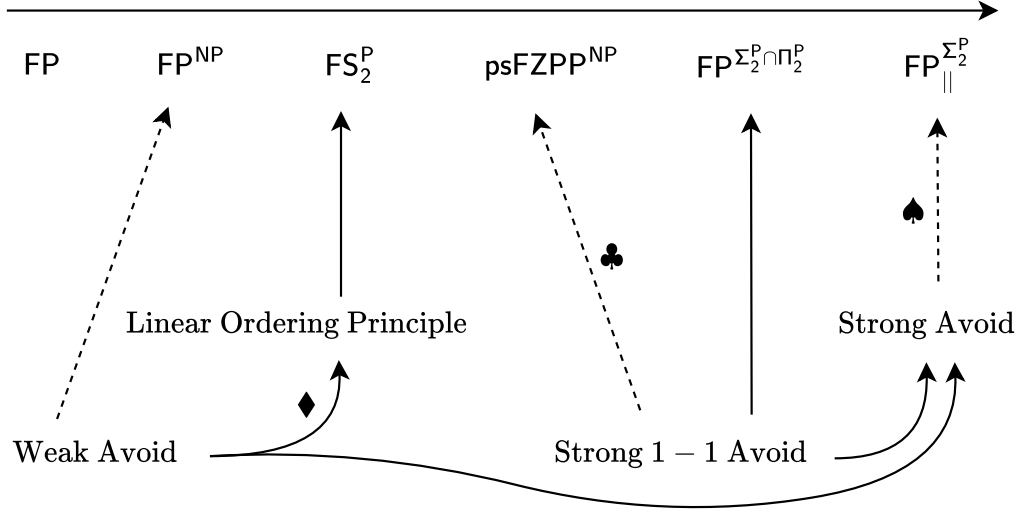


Figure 1.1: Inclusion diagram of relevant classes and search problems. Solid arrows represent the inclusion of the class at the base of the arrow into the class at its tip. Dotted arrows indicate non-inclusion of the base class into the tip class in the decision tree model. The main linear axis of classes along the top are all included from left to right, indicated by the long solid arrow above them. Our major results are the separations marked with ♠, ♣ and the inclusion marked with ♦. $\text{TFU}\Sigma_2^P$ essentially corresponds to $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$, see Lemma ??.

and at each leaf outputs a correct solution $y \notin \text{range}(f)$. Our lower bound can be interpreted as saying that if such a decision tree is forced to be non-adaptive, then either the circuits it queries at each step must grow to exponential size, or else the number of queries must grow to $N^{\Omega(1)}$. This also contrasts the situation with Weak Avoid, where as mentioned above, Li’s construction gives depth-3 AC_0 circuits of size $N^{O(\log N)}$ for solving Weak Avoid.

We believe our lower bound is of independent interest in circuit complexity. In particular, we give a very precise depth-3 lower bound for a *total search problem*. It is of course possible to construct contrived examples of total search problems which are hard for AC^0 circuits, for example “given $x \in \{0, 1\}^n$ output $b \in \{0, 1\}$ such that $\text{Parity}(x) = b$,” in such examples we can derive hardness of the search problem *by reduction from a decision problem*. In contrast, the lower bound we show here *cannot be established by reduction from any decision problem*. This follows from the fact that any decision problem which is reducible to a $\text{TF}\Sigma_2^P$ search problem lies in $\Sigma_2^P \cap \Pi_2^P$. In the decision tree model, this means that any language which can be solved by a small depth decision tree querying instances of Strong Avoid has both Σ_2^P and Π_2^P circuits. However, the lower bound we

are trying to show for Strong Avoid rules out the existence of any $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$ algorithm for Strong Avoid. Phrased more succinctly, our lower bound establishes that Strong Avoid is harder than *any* decision problem which can be reduced to it, which by definition means we cannot establish the lower bound itself by reduction from a decision problem. In light of this, to prove Theorem ?? we must develop new AC^0 lower bound techniques which are specially tailored to total search problems.

Our second main lower bound (Theorem ??) places Strong 1-1 Avoid outside of FBPP^{NP} . Recall that Strong 1-1 Avoid has the property of having *essentially unique solutions* and is thus reducible to a problem in $\text{TFU}\Sigma_2^P$. We will see (Lemma ??) that this means it is reducible to a *decision problem* in $\Sigma_2^P \cap \Pi_2^P$. Combining this with the above lower bound we obtain Theorem ??, which separates the decision tree class $\Sigma_2^P \cap \Pi_2^P$ from BPP^{NP} and in particular from S_2^P . This is in contrast to the situation for $\text{NP} \cap \text{coNP}$, which is known to collapse to P in the decision tree model. This improves a previous result of Fortnow and Yamakami [9] who showed that $\Sigma_2^P \cap \Pi_2^P \not\subseteq P^{\text{NP}}$ in the decision tree model.

1.3.2.1 Lower Bound Methods

A main technical ingredient in both lower bounds is a new Switching Lemma (Lemma ??) specialized for Avoid. Switching lemmas have been used for both circuit lower bounds for computing functions, and in proof complexity to prove lower bounds on the size of proofs of hard tautologies (e.g., [10, 11, 12]). Between the two, our argument bears a stronger resemblance to the second, however there are some key conceptual differences. The basic idea behind all switching lemmas is to show that under a random restriction ρ (from a suitable distribution) a low-width DNF is likely to be represented by a low-depth decision tree. Since a low-depth decision tree representation for a function f implies that both f as well as its negation can be represented by low-width DNFs, this in turn allows us to collapse an AND of low-width DNFs into a single low-width DNF, thus reducing the circuit depth by 1.

A major difference between various switching lemmas is the choice of distribution over restric-

tions, and the way in which the decision tree *represents* a DNF. In the original Switching Lemmas used to prove AC_0 lower bounds for parity, the restrictions are simply uniformly random partial restrictions, and the notion of represents is with respect to *every* input. That is, the decision tree computes the same function as the DNF.

In the case of switching lemmas used in proof complexity to prove AC^0 -Frege lower bounds for the $[N] \rightarrow [N + 1]$ pigeonhole principle, we think of the input as specifying a purported 1-1 function from $[N]$ to $[N + 1]$ (which cannot actually exist if N is finite). The chosen distribution over restrictions are partial 1-1 matchings from $[N]$ to $[N + 1]$, and a low-depth “PHP decision tree” in this context can make queries to a pigeon or to a hole at each vertex, and every path in the tree corresponds to a partial matching. Since, in reality, no 1-1 function from $[N]$ to $[N + 1]$ exists, these trees do not represent the original DNF in any standard way. However, if the input variables instead corresponded to a total 1-1 assignment from $[N]$ to $[N]$ (which do exist), then we can apply the same PHP Switching Lemma to prove that under a random partial 1-1 restriction, a low-width DNF is likely to convert to a low-depth PHP decision tree, which now represents the DNF in the sense that it agrees with the DNF on all input assignments that correspond to 1-1 mappings from $[N]$ to $[N]$.

In the case of Avoid, we have to modify the way of constructing a decision tree associated with a DNF so that the decision tree represents the original DNF in the sense that they are truth-functionally equivalent with respect to all 1-1 input functions from $[N]$ to $[M]$, where now M is strictly larger than N . To achieve this, we modify the notion of pigeonhole decision trees as follows. As in the original PHP Switching Lemma in proof complexity, each node of our decision tree will query either a pigeon or a hole. When a pigeon is queried at a node, we allow edges for all possible holes that it could be mapped to. But when a hole is queried, now we have to allow for the possibility that this hole is unmapped: in addition to allowing edges for each pigeon that could map to this hole, we allow an extra edge corresponding to the case where nothing maps to this hole. With this modification, our pigeonhole decision trees will represent the original DNF with respect to all 1-1 inputs from $[N]$ to $[M]$.

Another crucial distinction is how we use the Switching Lemma to reduce the depth of the circuit by one. In the proof complexity setting for the pigeonhole principle lower bounds, we think of N as infinite, and therefore with respect to 1-1 inputs, a DNF can be written as a low-width *matching disjunction*, where each term in the disjunction corresponds to a partial 1-1 function (or matching) from $[N]$ to $[N]$. After applying the PHP Switching Lemma and a union bound, each matching disjunction f (under ρ) becomes a low-depth “matching decision tree”, enabling a reduction in the overall circuit depth by one. To summarize, in the classical PHP Switching Lemma, the underlying depth-2 subcircuits are always low-width matching disjunctions, both before and after each application of the PHP Switching Lemma. In our case, the underlying depth-2 subcircuits are not of the same type before and after applying our Pigeonhole Switching Lemma; a consequence is that our Switching Lemma *cannot be applied twice*. This is not a defect of our method, but rather a necessary feature of any technique here, since our search problem *can be solved* by circuits of one higher depth. More specifically, we show that initially the bottom depth-2 subcircuits of C can be expressed as low-width matching disjunctions. But after applying the Pigeonhole Switching Lemma, and subsequent depth reduction, the new depth-2 subcircuits become *hole disjunctions* which are a generalization of matching disjunctions, where now each term in the DNF can specify not only a partial 1-1 matching, but also a subset of holes that are unmapped. With these appropriate modifications, our proof of the switching lemma is similar to previous proofs.

Equipped with the Pigeonhole Switching Lemma, we can give the high level view of both proofs. We start with Theorem ???. To prove Theorem ??, we would like to restrict attention to the class of 1-1 input functions from $[N]$ to $[N + 1]$. However if we truly restrict ourselves to 1-1 functions, then a lower bound is not possible by Lemma ???: for these inputs, there is a unique solution, and therefore a polynomial-size depth-3 circuits can easily check whether the unique solution has its i^{th} bit equal to 1 or to 0 and hence solve the problem unconditionally. To circumvent this barrier, we will prove a strengthening of Theorem ??, by giving a lower bound for Avoid on input functions $f : [N] \rightarrow [M]$, where M is larger than $N + 1$. By enlarging the range of f , we can focus our attention of f 's that are 1-1 since now for every input, there are at least $M - N$

distinct solutions. Note that this implies the lower bound stated above (with $M = N + 1$), since there is a direct reduction from Avoid on instances $[N] \rightarrow [M]$ to instances $[N] \rightarrow [N + 1]$: we simply map every element of $[M] \setminus [N + 1]$ to the element $N + 1$. Observe that this reduction does not preserve injectivity.

Now assume there exists small-size s depth-3 circuit C computing Strong Avoid on 1-1 functions from $[N]$ to $[M]$, $M \gg N$. We can first apply a standard argument (the Width Reduction Lemma ??) so that we can assume that the bottom-level fanin of C is at most $O(\log s)$. After this step, we can assume that C is a size- s , depth-3 circuit, where the bottom depth-2 subcircuits are low-width matching disjunctions. Next we apply our Pigeonhole Switching Lemma (as discussed above) which will guarantee that there exists a matching restriction ρ such that under ρ , all depth-2 matching disjunctions in C will convert to low-depth pigeonhole decision trees. This will allow us to reduce the overall circuit depth by 1, and afterwards each output bit of C will be computed by a low-width *hole disjunction*. As discussed above, a hole disjunction is a type of DNF that generalizes matching disjunctions: each term t in the hole disjunction can be viewed as partial information about the input f . The partial information consists of two parts: (i) first, t_1 specifies a small partial matching, pairing up some pigeons in $[N]$ to some holes in $[N + 1]$; (ii) secondly, t_2 specifies a small set of holes (disjoint from the holes mentioned in t_1 that are not in the range of f). It remains to prove a lower bound for circuits C for solving Avoid, where each output bit is specified by a low-width hole disjunction. This is also accomplished using a kind of restriction, but rather than choosing it at random we apply a careful deterministic process involving a novel covering argument. This step is somewhat reminiscent of early proofs of the Switching Lemma (e.g., [13, 14]).

The proof of Theorem ?? again uses the Pigeonhole Switching Lemma, together with a direct argument. We want to prove depth lower bounds for FBPP^{NP} , which informally are randomized decision trees of small height which, instead of querying variables, are allowed to query the value of an arbitrary low-width DNF over the inputs. By Yao's minimax principle, it suffices to prove that any low-depth P^{NP} decision tree cannot solve Avoid with probability $2/3$, with respect to

the uniform distribution of 1-1 functions. We think of this distribution in the following way: first sample a uniform *partial* 1-1 assignment ρ , then sample a uniform extension of ρ to a total assignment. Applying our Pigeonhole Switching Lemma and a union bound, we can argue that with high probability over the first choice of ρ , all of the NP queries in our P^{NP} decision tree T can be simplified to small depth pigeonhole decision trees, which overall allows T to be replaced by a low depth pigeonhole decision tree. It then remains only to argue that a pigeonhole decision tree of low depth cannot solve 1-1 Strong Avoid with non-trivial probability on a uniform extension f of ρ , which can be accomplished with a direct argument.

1.3.3 Linear Ordering Principle

We now discuss in more detail our results on the newly defined Linear Ordering Principle problem, abbreviated LOP. Recall that our main result here is Theorem ??, which says that Weak Avoid is polynomial time reducible to LOP. The proof follows much of the high level structure of Li's result placing Weak Avoid in FS_2^P , with some key modifications. Roughly speaking, Li's proof shows that given an instance of Avoid $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$, we can define a comparison relation $\sqsubset$ on $\{0, 1\}^{\text{poly}(n)}$, so that for some unique distinguished element $\pi^* \in \{0, 1\}^{\text{poly}(n)}$ we have $\pi^* \sqsubset \pi$ for all $\pi \neq \pi^*$, and π^* contains a solution to the original Avoid instance. In our case we need to define a similar comparison relation, which in addition *globally* acts as a total order on $\{0, 1\}^{\text{poly}(n)}$. To explain the argument more clearly we split the reduction into two parts. We first introduce an intermediate search problem called Forest Termination and reduce Weak Avoid to this problem, then we reduce Forest Termination to LOP. We note that our proof, as well as Li's, also bears a strong resemblance to the work of [15] who gave the first proof of the weak pigeonhole principle in the bounded arithmetic theory T_2 .

Our subsequent results show some appealing structural properties of the complexity class defined by reducibility to LOP. We start by proving closure under a broad class of reductions:

Theorem 1.9. *Any search problem which has a polynomial time P^{NP} Turing reduction to LOP also has a polynomial time many-one reduction to LOP.*

Combining this closure property with the fact that LOP has essentially unique solutions, we are able to conclude that LOP is *equivalent* in complexity to a decision problem. In particular we can define a decisional complexity class L_2^P for which LOP is the “complete problem” (despite being a search problem and not a language). We start by presenting a machine-based definition of L_2^P :

Definition 1.10. A language L is in the complexity class L_2^P if there is a polynomial time relation $R : (\{0, 1\}^*)^3 \rightarrow \{0, 1\}$ and a polynomial p , so that for all x , $R(x, \cdot, \cdot)$ defines a total order on $\{0, 1\}^{p(n)}$ whose minimal element a has $a_1 = L(x)$.

We then have the following equivalent characterizations:

Theorem 1.11. *The following are equivalent for a language L :*

1. $L \in L_2^P$
2. L is polynomial time many-one reducible to LOP.
3. L is P^{NP} -Turing reducible to LOP.

Conversely, the search problem LOP is polynomial time truth table reducible⁴ to a language in L_2^P .

We mentioned in passing before that LOP is easily shown to lie in the class FS_2^P , the functional analogue of the decision class S_2^P (this will be shown in Lemma ??). The same reason shows that $L_2^P \subseteq S_2^P$. For those unfamiliar with the somewhat unconventional class S_2^P , this is a complexity class introduced independently by Russell-Sundaram and Canetti [16, 17]. Their goal was to identify the smallest class in the polynomial hierarchy which is sufficient to capture randomized algorithms, in particular BPP and MA. Beyond this purpose the class rarely appears, and so far no natural problems have been exhibited which lie in S_2^P and not one of its more traditional subclasses (such as BPP or NP). In addition it seems that S_2^P does not have a complete problem, due to its definition involving a promise. We have identified here a subclass L_2^P of S_2^P , which is characterized exactly by a simple and natural total search problem, and which nonetheless maintains the interesting properties that motivated the original definition of S_2^P :

⁴This is essentially the most restrictive reduction possible when reducing a search problem to a language.

Theorem 1.12.

1. $P^{NP} \subseteq L_2^P$ and $BPP \subseteq MA \subseteq L_2^P$
2. $L_2^{E^5}$ contains a language of circuit complexity $2^n/n$.

In each case, the result stated for L_2^P was previously known to hold for S_2^P and is now shown to be inherited by the more natural subclass L_2^P . The only interesting property that is known of S_2^P which we were unable to prove for L_2^P is the Karp-Lipton theorem; we discuss this further Sections ?? and ??.

1.3.4 Open Problems

TODO: mention Volkavich stuff here, and Noah's paper We conclude our introduction with a few interesting problems which remain open. The first is rather broad:

Problem 1.13. *Show any additional inclusions or black-box separations which are not implied by the arrows in Figure ??.*

We specifically highlight the following:

Problem 1.14. *Is there an FP^{NP} Turing reduction from LOP to Weak Avoid?*

Our interest in this problem is the following observation:

Observation 1.15. *Say that Linear Ordering Principle is FP^{NP} Turing reducible to Weak Avoid. Then there is a particular language L of circuit complexity $\geq 2^n/n$, and a deterministic NP oracle algorithm $\mathcal{A}$ running in time $2^{O(n)}$, such that given oracle access to any language L' of circuit complexity $2^{\Omega(n)}$, $\mathcal{A}^{L'}$ computes L .*

This follows by composing the reduction of Weak Avoid to LOP, which produces a unique solution, with the (purported) second reduction from LOP back to Weak Avoid. Such a consequence would be rather surprising and interesting purely from the perspective of circuit complexity.

Next we highlight the problem of better clarifying the relationship between L_2^P and S_2^P :

⁵ L_2^E is the exponential-time analogue of L_2^P , where we replace “polynomial time” with $2^{O(n)}$ time” in its definition

Problem 1.16. Does $L_2^P = S_2^P$? Can they be separated in the decision tree model? Does L_2^P satisfy a Karp-Lipton theorem?

For this problem, we would say L_2^P “satisfies a Karp-Lipton theorem” if one could unconditionally prove the implication “ $NP \in P/poly \rightarrow PH = L_2^P$.” A notable property of S_2^P is that it is the smallest complexity class for which this statement is known to hold.

The next problem we highlight is in the realm of depth 3 circuits:

Problem 1.17. Does Weak Avoid have depth 3 circuits of polynomial size?

Recall that Li’s upper bound is only quasipolynomial, of size around $N^{\log N}$. This problem seems intimately connected to the long-standing open question in proof complexity of whether the Weak Pigeonhole Principle has polynomial size bounded depth Frege proofs; a *quasipolynomial* upper bound was shown by Paris Wilkie and Woods [15] using a very similar technique to Li’s, and [18] give a different bounded-depth Frege upper bound of lower depth, but still quasipolynomial size. Despite the strong aesthetic similarities we do not know a formal connection in either direction between these problems.

Lastly, the search problems discussed here have other connections to bounded arithmetic. In particular, the LOP principle has been investigated in several papers, defined explicitly for the first time in [7] and studied mostly within the context of characterizing the strength of Jerebek’s bounded arithmetic theory of approximate counting relative to weaker theories, and also as a new avenue for approaching the longstanding problem of separating Buss’ T_2 hierarchy by sentences of fixed complexity. Buss, Kolodziejczyk and Thapen [19] observe that the LOP principle is provable in both T_2^2 and in APC_2 , and ask whether or not LOP is provable in the weaker theory $T_2^1 + sWPHP$, where $sWPHP$ is the surjective weak pigeonhole principle, and corresponds to the search problem Weak Avoid. Atserias and Thapen [20] resolve this question, proving that in the relativized setting, $sWPHP$ does not prove the LOP principle over T_2^1 . In fact they prove a stronger result, that $sWPHP$ cannot prove the HOP principle over T_2^1 , where HOP is a Σ_1^b version of LOP. It seems possible that the techniques here could be used to give a negative answer to Problem ???. A relatively unexplored area that is likely to be fruitful is to discover more relationships

between natural search problems lying in the second level of the polynomial hierarchy (and higher) and corresponding systems of bounded arithmetic.

1.4 Derandomization from Time-Space Tradeoffs

In this chapter, as well as Chapter ??, we will give a deterministic polynomial time algorithm for solving a certain class of explicit construction problems. In both Chapters, the correctness of our algorithm will hinge on a certain complexity-theoretic *hardness assumption*, placing these results within the rich landscape of *hardness v.s. randomness* tradeoffs which have become ubiquitous in complexity theory since their introduction in [21] and subsequent generalized treatment in [22, 23] **TODO: reference a part of the introduction, or perhaps the first content chapter, which explains these.**

We motivate the algorithm in this chapter by a certain *counter-factual* thought experiment. When an explicit construction problem is defined, we have a nonconstructive proof that it is total; our starting point is to imagine a world in which some explicit construction problem is *not solvable in polynomial time*, i.e. the nonconstructivity of said proof is *inherent*. When a particular theorem appears to be inherently non-constructive, in that no polynomial time algorithm can witness the solutions it guarantees, one perspective we can take is that this theorem is “effectively false” in a certain context, despite being unconditionally true in general. This is essentially the outlook presented by Yao in his seminal paper introducing the basis of theoretical cryptography [21]. Here, Yao focuses on the central tenets of Shannon’s information theory. He argues that although Shannon’s theorems are of course provably correct, in many scenarios it appears computationally infeasible to witness their truth. For example, an output of some function $f: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ under the uniform distribution on $\{0, 1\}^n$ has entropy n , and thus by a result of Shannon can be encoded on average using n bits. However for an observer who sees only the outputs of this distribution, there seems to be no efficient way to generate such a code without the ability to efficiently invert f . If we posit that there are some specific functions f for which it is in fact impossible to efficiently realize Shannon’s theorem in this sense, one might venture to say that Shannon’s theo-

rem *effectively fails* for f in the computational realm, perhaps allowing us to carry out tasks which would, in the absence of computational constraints, be deemed impossible. Indeed it is widely conjectured that there are efficiently computable functions f of this form, and this conjecture underpins the security of many cryptographic protocols. Similar situations are abundant in the field of cryptography, where the computational infeasibility of *witnessing* impossibility theorems from information theory allows us to effectively bypass them.

With this perspective in mind, we turn our attention to the following kind of explicit construction problem. Suppose that there is a pair of efficiently computable functions $\mathcal{G} = \{g_n: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}\}_{n \in \mathbb{N}}$, $\mathcal{F} = \{f_n: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n\}_{n \in \mathbb{N}}$, and our explicit construction problem is: given 1^n , find some n -bit string x such that $f_n(g_n(x)) \neq x$ in $\text{poly}(n)$. Note that this does not quite capture explicit construction problems reducible to range avoidance, since rather than a near length-extending map g_n we are also given its supposed “inverse” f_n . Now, if this explicit construction problem is not efficiently solvable, then we have found a pair of efficiently computable functions $\mathcal{F}, \mathcal{G}$, such that no polynomial time algorithm is able to construct an n -bit string x such that $f_n(g_n(x)) \neq x$ in $\text{poly}(n)$ time. In this hypothetical world where $\mathcal{F}, \mathcal{G}$ are an “effective counterexample” to the weak pigeonhole principle, we have access to an efficient compression scheme which allows us to encode any n -bit string using $n - 1$ bits. What improbable feats can be accomplished in such a world?

In this work we give one answer to this question: if the weak pigeonhole principle *effectively fails* in the above sense, we can utilize this failure to construct an efficient data structure which allows us to simulate RAM computations in low space and near-linear time on a 1-tape Turing machine. Our simulation proceeds by bootstrapping our compression scheme that saves 1 bit, into a *dynamic* compression scheme that can store an m -bit memory in m^ϵ bits, such that each cell of memory can be read or updated in m^ϵ time. Stated in contrapositive, if we assume mild time-space tradeoffs for simulating RAM machines on a 1-tape Turing machine, we obtain a generic deterministic algorithm to witness the weak pigeonhole principle for any pair of maps $\mathcal{F}, \mathcal{G}$.

TODO: say something about the problem LOSSY, a more lasting contribution of this paper,

along with the R-LOSSY thing...

1.4.1 Our Contributions

1.4.1.1 Derandomization from Time-Space Tradeoffs

Let $C, \mathcal{D} = \{C_n: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}\}_{n \in \mathbb{N}}, \{D_n: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n\}_{n \in \mathbb{N}}$. We call such a pair a “uniform compression scheme,” where C is the “compressor” which encodes n -bit “messages” by $n - 1$ -bit “codewords,” and $\mathcal{D}$ is the “decompressor” which maps $n - 1$ -bit codewords to the messages they represent⁶. We use the term “uniform” since in the relevant cases, C and $\mathcal{D}$ will each be described by a single Turing machine which computes C_n (resp. D_n) for all n . We will say that a string $x \in \{0, 1\}^n$ is “incompressible” for such a scheme if $D_n(C_n(x)) \neq x$, i.e. if $\mathcal{D}$ cannot be used to recover the message x from the codeword assigned to it by C . In this chapter, we study the complexity of the explicit construction problem of generating incompressible strings for uniform compression schemes. This task can be viewed naturally as a derandomization problem, since a randomly chosen n -bit string will be incompressible with high probability with respect to any fixed compression scheme.

Our results then show how to derive efficient derandomized algorithms for generating incompressible strings, assuming certain *uniform* lower bounds. The lower bounds we consider are time-space tradeoffs for simulating RAM by 1-tape machines. Roughly, they posit that there are problems solvable in time T on a RAM machine which cannot be solved on a 1-tape machine using $T^{0.01}$ space and $T^{1.01}$ time. Depending on certain features of the compression scheme and the resources available to the explicit construction algorithm, the specifics of the tradeoff assumption will vary, as we shall explain in Figure ??.

Initially, we consider the case of “poly-time compression schemes”, where both C and $\mathcal{D}$ are computable in polynomial time. We show that for such schemes, incompressible strings can be constructed deterministically in polynomial time assuming time-space tradeoffs for deterministic exponential time:

⁶The formal definition given in Section ?? is slightly more general, but we will focus on this special case for now.

Theorem (Theorem ??). *Suppose there is some exponential time bound $T(n) \geq 2^{\Omega(n)}$ and some $\epsilon > 0$ such that it is impossible to simulate $T(n)$ -time RAM computations on 1-tape Turing machines that use $T(n)^{1+\epsilon}$ time and $T(n)^\epsilon$ space. Then for any poly-time compression scheme, there is a polynomial time algorithm that produces incompressible n -bit strings for this scheme on input 1^n (for infinitely many n).*

Constructing incompressible strings for poly-time compression schemes is a natural and quite broad derandomization problem. Theorem ?? gives a novel connection between derandomizing this task and proving *uniform* time-space tradeoffs— in contrast, the assumptions previously required to derandomize this sort of problem require a lower bound against *non-uniform* algorithms, such as circuits. To put this derandomization task in a more familiar context, in Section ?? we show that several well-studied explicit construction problems, such as the construction of large prime numbers or the construction of truth tables of high circuit complexity/formula size, can be reduced to the problem of finding incompressible strings for some uniform compression scheme. In each case, the compression scheme will either be computable efficiently, or efficiently with access to an oracle for some search problem in **FNP** which is not known to be **NP**-complete.

To state our other main results in their most interesting form, we focus our attention now on the construction of strings/truth tables of high complexity with respect to some non-uniform complexity measure, such as formula size, circuit size, or time-bounded Kolmogorov complexity. In each case there is an associated **NP** search problem, where we are given a string/truth table and must find a small formula/circuit/program computing it (if one exists); we call this the “compression problem.” The following table shows how our three main theorems relate various time-space tradeoff hypotheses to such problems:

Hypothesis: For some exponential time bound T and some $\epsilon > 0...$	Implication :
$\text{RAM-TIME}[T] \not\subseteq \text{1-TISP}[T^{1+\epsilon}, T^\epsilon]$	If the compression problem has a polynomial time algorithm, then incompressible strings can be constructed in polynomial time.
$\text{NTIME}[T] \not\subseteq \text{1-NTISPG}[T^{1+\epsilon}, T^\epsilon, T^\epsilon]$	Incompressible strings can be constructed in polynomial time with an NP -oracle. In particular, $\mathbf{E}^{\text{NP}} \not\subseteq \text{size}[2^n/2n]$.
$\text{RAM-TIME}[T] \not\subseteq \text{1-NTISPG}[T^{1+\epsilon}, T^\epsilon, T^\epsilon]$	Incompressible strings can be constructed in polynomial time with an oracle for the compression problem.

Figure 1.2: Main Results. We use the shorthand $T = T(n)$.

The classes seen on the left hand side will be defined formally in Section ??, but we give a brief explanation here so that the table can be interpreted appropriately. $\text{TIME}[T(n)]$ and $\text{NTIME}[T(n)]$ are defined in the standard way using multitape machines; the prefixes 1 and RAM indicate, respectively, either a restriction of the model to 1-tape machines or a strengthening to random access machines. The class $\text{TISP}[T(n), S(n)]$ consists of problems decidable simultaneously in time $T(n)$ and space $S(n)$. Finally, $\text{NTISPG}[T(n), S(n), G(n)]$ consists of problems decidable by a nondeterministic machine running in time $T(n)$ which, on every computation path, uses at most $S(n)$ space and $G(n)$ nondeterministic guesses.

For a concrete example of how to apply these results, let's focus on the compression problem **CIRCUIT SYNTHESIS** (given a truth table find a small circuit for it if one exists), and the explicit construction problem of producing truth tables of exponential circuit complexity. The first hypothesis in the above table implies that if **CIRCUIT SYNTHESIS** is in **P** then there is a polynomial time construction of hard truth tables (i.e. **E** has exponential circuit complexity). The second hypothesis implies unconditionally that there is an efficient **NP** oracle construction of hard truth tables (i.e. $\mathbf{E}^{\text{NP}}$ has exponential circuit complexity). Finally, the third and strongest hypothesis tells us that there is a polynomial time construction of hard truth tables using an oracle for **CIRCUIT SYNTHESIS**.

SIS.

The case of large prime construction does not fit in as neatly to the above picture, as the scheme we devise for this problem will require a factoring oracle for both the compressor and decompressor (while the above problems have a polynomial-time computable decompressor). In this case we get the following result:

Theorem (Corollary ??). *Under the hypothesis in the top row of the above table, a polynomial time algorithm for factoring implies a polynomial time algorithm to construct $32n$ -bit primes of magnitude $> 2^n$ for infinitely many n .*

If we forgo the questionable assumption that factoring lies in P, we get:

Theorem (Corollary ??). *One of the following is true:*

1. *For every exponential time bound T and every $\epsilon > 0$, every language decidable in time $T(n)$ on a RAM machine can be decided in time $T(n)^{1+\epsilon}$ and space $T(n)^\epsilon$ by a 1-tape machine with a factoring oracle, which makes oracle calls of length at most $T(n)^\epsilon$.*
2. *There is a polynomial time algorithm with a factoring oracle that generates $32n$ -bit primes of magnitude $> 2^n$ for infinitely many n .*

The problem of deterministically generating large primes has been investigated previously in several works, and was notably the subject of the Polymath 4 project [24]. In the public discussion forums for this project, it was explicitly asked whether a polynomial time algorithm for factoring, or more generally an oracle for factoring, would help. More recently, Oliveira and Santhanam gave a subexponential time “pseudodeterministic” construction of large primes [25], using only the fact that primality is testable in P [26] and that primes occur with non-negligible frequency.

1.4.1.2 BPP and the Weak Pigeonhole Principle

In Section ?? we briefly consider the relationship between various search problems associated with the weak pigeonhole principle, and the “full derandomization task” characterized by the class

prBPP. Observe that the problem introduced above of finding incompressible strings for uniform compression schemes can be generalized to a TFNP search problem – instead of considering compression schemes generated by uniform Turing machines, we can consider the search problem where a compression scheme of some fixed length is given as input in the form of a pair of boolean circuits:

Definition 1.18. In LOSSY CODE, we are given as input a pair of circuits $C: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$, $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, and must output some $x \in \{0, 1\}^n$ such that $D(C(x)) \neq x$.

While finding a deterministic algorithm for LOSSY CODE appears to be a quite generic derandomization problem, it would be a major breakthrough to show that LOSSY CODE captures the “full derandomization problem:” since LOSSY CODE lies in TFNP, if prBPP reduces to LOSSY CODE then $\text{BPP} \subseteq \text{NP}$, which is a notorious open problem. In Section ??, we show that a natural generalization of LOSSY CODE, where we allow the compressor C to be randomized, is indeed strong enough to characterize prBPP precisely. The formal problem is as follows:

Definition 1.19. In R-LOSSY CODE, we are given as input circuits $C: \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^{n-1}$, $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, and must find some x such that $\Pr_r[D(C(x, r)) = x] < \frac{1}{2}$.

In this work we demonstrate:

Theorem (Theorem ??). *R-LOSSY CODE is complete for prBPP under deterministic Turing reductions.*

The relation between pigeonhole principle search problems and more standard derandomization problems is summarized in Figure ??.

The proof of Theorem ?? is mostly standard, following quite directly from Yao’s next bit predictor lemma. From the proof of this theorem we are able to extract the following interesting corollary:

Corollary (Corollary ??, Informal). *If the fixing of the leftover bits in Yao’s hybrid argument can be derandomized, then $\text{BPP} \subseteq \text{NP}$. Indeed, under this assumption, every problem in prBPP reduces to the search problem LOSSY CODE in $\text{PPP} \subseteq \text{TFNP}$.*

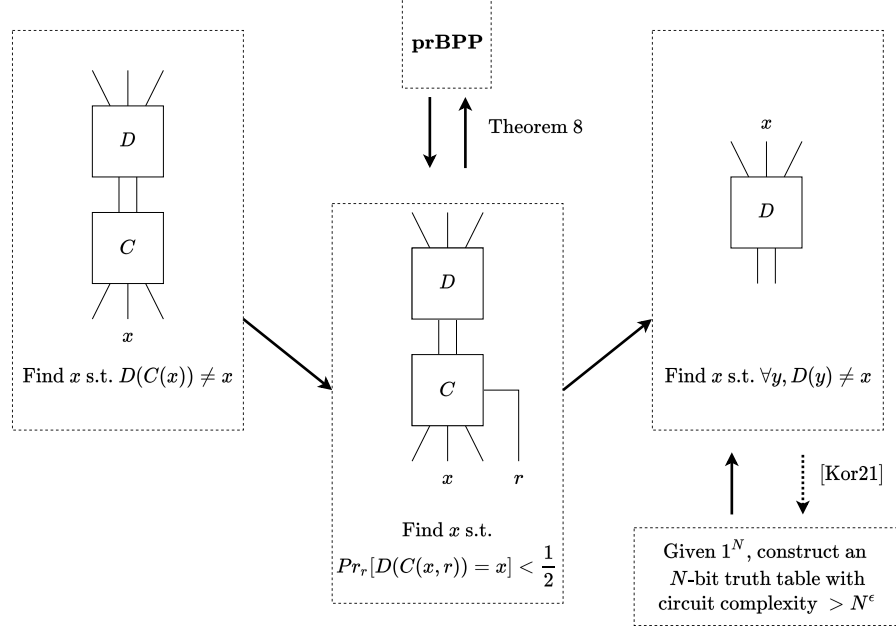


Figure 1.3: Relations between the total search problems associated with various weak pigeonhole principles, and standard derandomization problems. Solid arrows represent deterministic polynomial time reductions, while dotted arrows represent NP-oracle reductions.

In other words, derandomizing a particular step in Yao’s classical argument implies a quite universal derandomization of prBPP.

1.4.2 Relation to Prior Work

Hardness vs. Randomness As mentioned above, the task of constructing incompressible strings can be naturally viewed as a derandomization problem, and is thus amenable to the standard hardness/randomness paradigm. In particular, when the compression scheme is polynomial time computable as in Theorem ??, the standard hardness assumptions used to derandomize BPP (e.g. [23]) would suffice to yield a polynomial time construction of incompressible strings. Such hardness assumptions require a lower bound for a language in E against some non-uniform model of computation, such as boolean circuits. In contrast, Theorem ?? gives a hardness/randomness connection for this problem which only requires a *uniform lower bound* for a language in E, in particular a lower bound against low-space algorithms running in slightly more time using a weaker memory model.

The Easy Witness Lemma: Perhaps the closest prior work to our results is the “Easy Witness Method,” initiated by [27] and furthered in [28] and [29], which roughly says that assuming E^{NP} has small circuits, any nondeterministic exponential time computation must have witnesses of low circuit complexity. This immediately implies that unless E^{NP} requires large circuits, we can efficiently simulate $NTIME[2^n]$ using limited nondeterminism by “guessing a small circuit.” In Appendix ??, we show that this old method (along with one other well-known tool) is in fact enough to prove the second implication in Figure ??, although this connection to time-space tradeoffs does not seem to have been noted previously. However, this proof heavily utilizes the distinctive power of nondeterminism, whereby additional “guesses” allow us to vastly simplify the verification procedure, and does not seem to extend to our other two main results.

1.4.3 Known Time-Space tradeoffs:

Finally, we cover the known results on time-space tradeoffs and separations between the RAM and 1-tape models. We first emphasize the following: all three time-space tradeoff hypotheses stated on the left-hand side of Figure ?? are known to hold *unconditionally* when the time bound T is $O(n)$. In particular, an old result of Maass [30] shows that the set of palindromes is recognizable in quasilinear time on a deterministic RAM machine, but requires $\Omega(n^2)$ time on a nondeterministic 1-tape machine. However, Maass’s proof is essentially a counting argument which crucially relies on the entropy of the input being comparable in magnitude to the total computation time, which is no longer the case for exponential time computations. When it comes to separating the RAM and 1-tape models for generic time bounds, a result of [31] shows that there must be *some* slowdown when simulating a RAM on a 1-tape machine for *any time bound* greater than $n \log \log n$, but the slow-down is an astronomically small multiplicative factor, on the order of $\log \log T(n)$.

Another sequence of works ([32], [33], [34] among others) has shown unconditionally that nondeterministic linear time cannot be solved in $TISP[n^{1+\epsilon}, n^\epsilon]$ for certain fixed values of $\epsilon > 0$, even when the simulating machine is given access to RAM. These results can be scaled to larger time bounds as follows: for any time-constructible T , $\Sigma_2 TIME[T(n)]$ is not contained

in $\text{TISP}[T(n)^{1+\epsilon}, T(n)^\epsilon]$ for certain fixed values of $\epsilon > 0$. Such results are obtained by combining hierarchy theorems with a fast simulation result, whereby a Σ_2 machine can simulate a $\text{TISP}[T(n)^{1+\epsilon}, T(n)^\epsilon]$ computation in time $T(n)^{1-\delta}$ for some fixed $\epsilon, \delta > 0$. It is evident that such results will not be sufficient for our purposes, as they only rule out efficient low-space simulations of Σ_2 computations by deterministic machines.

1.4.4 Main Tool: the J-tree

TODO: update the terminology here to match other parts of paper? The basis of our main proofs is a construction which we call the “J-tree.” Roughly speaking, the J-tree is a data structure which allows us to store an array of T elements, subject to fast update/query operations, using significantly less than T bits. While this task is information-theoretically impossible (by the weak pigeonhole principle), the J-tree uses a compression scheme C, D to perform its operations, and whenever it fails in its role as a data structure, it is able to print out an incompressible string for this scheme. Thus, assuming no polynomial time algorithm can witness the weak pigeonhole principle for C, D , no polynomial time algorithm can witness the failure of this data structure. This will allow our low-space simulations to operate a RAM memory using low space on a 1-tape machine, in such a way that if the simulation fails, then a polynomial time algorithm can witness this failure, and thus witness the weak pigeonhole principle for C, D .

The core construction underlying the J-tree has an intriguing history dating back to the late 1970’s. In the course of a decade, this construction was used, seemingly independently, to prove three seminal results in cryptography and logic:

1. In 1979⁷, Merkle showed how to iterate a cryptographically secure hash function $h: \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$ in a tree-like fashion to construct a secure digital signature scheme whose signature size increases logarithmically in the number of messages signed [35]⁸.
2. In 1986, Goldwasser, Goldreich and Micali showed how to iterate a pseudorandom generator

⁷Although the cited work is dated to 1988, Merkle filed a patent for the concept about a decade earlier.

⁸The Merkle Tree is in some sense “dual” to the other two examples, as it iterates a function which *compresses* strings by a factor of two, while the others iterate a function that *extends* by a factor of two.

$G: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ in a similar tree-like manner into a family of functions $\{G_x: \{0, 1\}^n \rightarrow \{0, 1\}^n\}_{x \in \{0, 1\}^n}$ whose input/output behaviours are indistinguishable from random, assuming G is a secure PRG [36].

3. In 1988, Paris, Wilkie, and Woods gave a novel proof of the weak pigeonhole principle in bounded arithmetic[15]. Again, the same process of iterating a function $f: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ is applied. Here, the function f is assumed to be a bijection between $\{0, 1\}^n$ and $\{0, 1\}^{2n}$, and the iteration procedure is the first step towards establishing a contradiction in the relevant proof system.

Following [15], Jeřábek [37] used this same construction within bounded arithmetic to establish a deep connection between the weak pigeonhole principle and circuit complexity. This work most closely informs our perspective, hence the name “J-tree.” In [38], the techniques of Jeřábek were translated to the language of traditional complexity theory, which yield the following interpretation of this tree-like iteration process: given a circuit $D: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, we can efficiently construct a new circuit $D^*: \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$ satisfying the following: any string in the range of D^* is a truth table of low circuit complexity, and any string outside the range of D^* can be used to efficiently construct a string outside the range of D using an NP oracle. If we have a compression scheme $C: \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$, $D: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, the same argument lets us iterate these to a scheme C^*, D^* of message length $2^k n$ and code length n , where again D^* always outputs low complexity truth tables, and now we have that any incompressible string for C^*, D^* can be transformed in polynomial time (without an oracle) into an incompressible string for C, D .

Thus, if it is hard to witness the weak pigeonhole principle for C, D , then we will be able to take any string (“message”) of length $2^k n$ and find a small circuit computing it (“codeword”) by feeding it through C^* . A circuit can naturally be interpreted as a data structure storing an array subject to fast query operations: the truth table represents the contents of the array, and the circuit allows us to efficiently compute a particular bit in the truth table given its index. However, to use the J-tree to simulate RAM computations, we will also require fast update operations, which let us set a position in the array to a new value. Our main addition to this line of work is the “J-Tree

Update Lemma” (Lemma ??), which gives a procedure that accomplishes this second task. The Update Lemma does not seem to have an analogue in any of the previously mentioned works.

1.5 Stronger Cell Probe Lower Bounds via Local PRGs

We start by briefly introducing the three main subjects of this chapter: static data structure lower bounds, NC^0 pseudorandom generators, and range avoidance for NC^0 circuits.

Static Data Structure Lower Bounds: The cell probe model of data structures introduced in [39, 21] is a fundamental computational model in which to study tradeoffs between storage space and access time for information retrieval problems. In the *static* cell probe model, a data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ is fixed, where $[M]$ is an abstract set of “datapoints,” $[N]$ is an abstract set of “queries” one might wish to make about a data point, and $F(x, y)$ is the correct answer to the x^{th} query about the datapoint y . The goal is to preprocess any given datapoint $y \in [M]$ into a data structure $\mathcal{E}(y) \in \Sigma^S$ consisting of S “cells,” each cell holding a character from some finite alphabet Σ , so that later we may answer different queries $x \in [N]$ about y by reading at most t cells from the data structure $\mathcal{E}(y)$. S is called the “space complexity,” t the “time complexity,” and $w = \log |\Sigma|$ the “word-length” (number of bits to store the contents of a single cell). As a prototypical example (which our main lower bounds will apply to), let $d \leq n$ be given, $N = 2^n$, $M = 2^{\binom{n}{\leq d}}$. Identifying $[N]$ with $\mathbb{F}_2^n$, and $[M]$ with the set of degree $\leq d$ multilinear polynomials over $\mathbb{F}_2^n$, we can define the *polynomial evaluation problem* $\mathbb{F}_2\text{-Eval}_n^d : [N] \times [M] \rightarrow \{0, 1\}$ with $\mathbb{F}_2\text{-Eval}_n^d(x, p) = p(x)$; in this case to solve the data structure problem, we want a method for preprocessing polynomials p into small space data structures $\mathcal{E}(p)$, so that upon any query $x \in \mathbb{F}_2^n$ we may evaluate the polynomial p at the point x by querying only a few cells of the data structure.

The basic problem in static data structure lower bounds is to prove, for a particular problem F , that any data structure solution for F must use a large amount of either space, time, or word length (word length is generally of secondary importance). For *any explicit problem* $F : [N] \times [M] \rightarrow \{0, 1\}$, the best lower bound provable by techniques known prior to this work, for any $t > 1$ and

any setting of the word length w , is at best:

$$S \geq N^{\frac{1}{t}} \cdot (\log M)^{1-\frac{1}{t}} \quad (1.1)$$

Lower bounds of this kind were first achieved by [40] using a method that was rediscovered in later work and which is now known as *cell sampling*. In the special case of *nonadaptive bit probe data structures* (queries chosen nonadaptively, word length $w = 1$), a lower bound exceeding (??) has been shown in two recent works. First, [41] achieved a near-maximal space lower bound $S \geq \frac{N}{2}$ in the case $t = 2$ for a problem with $M \leq \text{poly}(N)$ datapoints. Subsequent work of [42] improved this to $S \geq (1 - \epsilon)N$ for arbitrarily small $\epsilon > 0$ (the lower bound is for a different explicit problem), and for all larger $t > 2$ proved a bound like ?? with the exponent $\frac{1}{t}$ replaced with $\frac{1}{(t-1)}$. In Section ?? we will discuss these works, as well as the broader history of static data structure lower bounds, in some more detail.

NC⁰ Pseudorandom Generators: A prominent line of work initiated in [43, 44] has studied the existence of cryptographic PRGs in NC⁰. An NC_t⁰-PRG is function $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $m > n$, so that: (1) each output depends on only t inputs, and (2) G is a cryptographic pseudorandom generator: no polynomial time algorithm can distinguish a random output of G from a truly random m bit string. We say that G is an NC⁰ generator if it is NC_t⁰ for some $t = O(1)$. The existence of such PRGs has been the subject of a great deal of work over the past two decades; we refer the reader to a comprehensive survey [45] and the references therein.

An important parameter is the *stretch* of the generator, which denotes the relation between n, m . We say that G has nontrivial stretch if $m > n$, and polynomial stretch if $m \geq n^{1+\Omega(1)}$. NC⁰ generators with nontrivial stretch can be shown to exist under standard number theoretic cryptographic assumptions [46], however their utility has so far been limited. On the other hand, NC⁰ generators with polynomial stretch are not known to exist under any more standard hardness assumptions, but have been found to have a wide range of advanced cryptographic applications [47], most notably in

the construction of Indistinguishability Obfuscation [48, 49]. For this reason, a great deal of work has gone into studying the existence of polynomial stretch NC^0 PRGs as a foundational assumption in its own right, and various algorithmic methods for distinguishing such PRGs have been studied. The current state of the art shows that an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ can be broken in polynomial time once $m \gg n^{\frac{t}{2}}$ ([50, 51, 52]).

NC^0 Range Avoidance: We finally turn to our third main subject of study, NC^0 range avoidance. The object in question here is the same as above: an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with $m > n$ and $t = O(1)$. However, instead of trying to distinguish the output of G from random, our goal now is to solve the Range Avoidance problem for G . [53] was the first to investigate the range avoidance problem for NC^0 circuits, and showed that a FP^{NP} algorithm for this problem would imply the breakthrough circuit lower bound $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$. The same question of *stretch* arises here: the aforementioned result of [53] requires us to solve instances with barely nontrivial stretch. On the other hand unconditional algorithms have been given once the stretch is sufficiently large, in particular once $m \gg \frac{n^{t-1}}{\log n}$ ([54, 55]). An important question posed in [53] is whether unconditional FP^{NP} algorithms for NC^0 range avoidance can be found in the general polynomial stretch regime.

The Main Correspondence: The starting point of our work is the observation that the 3 problems discussed above are tightly related to one another. The relation between NC^0 PRGs and NC^0 range avoidance is rather obvious (although still insufficiently explored prior to this work), and so the key point is the relation we observe between these two topics pertaining to NC^0 circuits on the one hand, and the project of proving cell probe lower bounds on the other. This relationship follows directly from the definitions of the objects in question, and is more appropriately understood as a *perspective shift* rather than a new result.

To state the connection in its simplest form, we focus our attention for now on a special setting of the cell probe model: the word length is 1 (i.e. $\Sigma = \{0, 1\}$, often called the *bit probe* model), and

the data structures in question are *nonadaptive*. Here, nonadaptive means that the data structure will decide which cells to probe as a function only of the query, in contrast to a general *adaptive* data structure which may decide on the next cell to probe based on the outcomes of previous probes. In this setting we have:

Observation. *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem, which we interpret as a matrix $F \in \{0, 1\}^{N \times M}$ with N rows and M columns below. The following are equivalent:*

1. *F does not have nonadaptive bit probe data structures with space complexity S and time complexity t .*
2. *The columns of F are a set of M strings $\mathcal{F} \subseteq \{0, 1\}^N$, $|\mathcal{F}| = M$, such that for any NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$, there exists a string $f \in \mathcal{F}$ with $f \notin \text{range}(G)$.*

Proof. Say that F has nonadaptive bit probe data structures of space S and time t . Then, to every $y \in [M]$ we may associate an encoding $E_y \in \{0, 1\}^S$, so that for every $x \in [N]$ we may determine $F(x, y)$ by probing (nonadaptively) t bits of E_y . Considering the y^{th} column of F as a string $f_y \in \{0, 1\}^N$, we see that there is a NC_t^0 function $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ so that $G(E_y) = f_y$ for every $y \in [M]$; in particular, the x^{th} output of G runs the data structure's nonadaptive query procedure corresponding to the query x . The same reasoning applies in the reverse direction. $\square$

In other words, finding an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ which requires large space for time t in the nonadaptive bit probe model is *exactly equivalent* to constructing explicit *list-solutions* for NC_t^0 range avoidance. By a list-solution, we mean a list of candidate strings, so that for *any instance* of NC_t^0 range avoidance G (with appropriate parameters), one of the strings in this list must be a range avoidance solution for G . We emphasize that an analogous connection exists for the general cell probe model (adaptive and with arbitrary word length), which is explained formally in Section ?? . The problem of finding list-solutions for NC^0 range avoidance was considered already in [54]. Indeed, via the above correspondence the results of [54] recover a nonadaptive bit-probe lower bound of [42] in a certain range of parameters.

The connection between NC^0 -PRGs and cell probe lower bounds is more difficult to state formally, but even more central to our main results. In one direction, we will show how to use the best known methods for distinguishing NC^0 PRGs to achieve state of the art lower bounds in the cell probe model; this is the most significant contribution in this work, and after stating our main results below we will describe this methodology in more detail in Section ?? (“Our Techniques”). In the other direction we will use the conjectured security of various NC^0 PRGs to exhibit a *natural properties barrier* for data structure lower bounds, in the spirit of Razborov-Rudich [56].

We note that connections of a similar flavor have been utilized in the literature on sampling lower bounds initiated by Viola [57]. Connections between restricted variants of range avoidance and data structures were first observed in [54], who used a cell probe data structure from [58] to develop more efficient reductions to range avoidance; this proof was our original inspiration to explore more deeply the connections between NC^0 range avoidance and the cell probe model.

1.5.1 Our Results

State of the Art Cell Probe Lower Bounds: Using the perspective shift introduced above, we are able to break a decades-old barrier in static data structure complexity, and prove cell probe lower bounds for a explicit problems which are quadratically better for space as a function of time than all previous methods. Our first lower bound holds any data structure problem supporting a k -wise independent distribution; this is the class same problems originally considered by [40] and in various follow up works (including [59, 60]), and we are able to obtain tight lower bounds across essentially the entire range of values $k \leq N$, provided the time complexity t is an even number:

Theorem 1.20. *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a k -wise independent distribution supported on its columns. Let $S, t, w \in \mathbb{N}$ be given with t an even number and assume $k > tw + 1$. Then, for any space S , time t , word length w cell probe data structure solving F we must have:*

$$1. \ S \geq \left(\frac{N}{\log N} \right)^{\frac{2}{t}} \cdot \left(\frac{k}{\log N} \right)^{1-\frac{2}{t}} \cdot t^{-1} 2^{-O(w)} \text{ for all } t \log N \leq k \leq N 2^{-O(tw)} t^{-\frac{t}{2}}$$

$$2. S \geq \left(\frac{N}{\log N} \right)^{\frac{1}{t(\frac{1}{2} + \frac{2}{k})}} \cdot t^{-1} 2^{-O(w)} \text{ for all } tw + 1 \leq k \leq t \log N$$

Remember that we often interpret $F : [N] \times [M] \rightarrow \{0, 1\}$ as a boolean matrix with N rows and M columns, which explains our use of the phrase “the columns of F .” If t is an odd number, the above lower bound holds with $t + 1$ in place of t ; hence this lower bound gives a polynomial improvement (for space as a function of time) over previous methods for all $t > 3$, gives a near-quadratic improvement for all even t , and approaches a quadratic improvement for large odd values of t . Using any of the various explicit examples of k -wise independent distributions with support size $N^{O(k)}$ (e.g. univariate polynomials over $\mathbb{F}_{2^n}$), we obtain explicit problems $F : [N] \times [M] \rightarrow \{0, 1\}$ satisfying the lower bound

$$S \geq \tilde{\Omega} \left(N^{\frac{2}{t}} \cdot (\log M)^{1 - \frac{2}{t}} \cdot 2^{-O(w)} \right)$$

across essentially the entire spectrum of nontrivial values $M \in \{1, \dots, 2^N\}$. We thus achieve a near-quadratic improvement in the state of the art explicit cell probe lower bounds across this entire spectrum of relations between M, N . We emphasize that, in addition to giving a quadratic space improvement over the *cell sampling* bound (the first improvement of any kind), our lower bound is also the first to achieve any polynomial improvement over the much simpler *communication bound* $S \gtrsim N^{\frac{1}{t}}$ for a problem with N queries and $M \leq \exp(N^{o(1)})$ datapoints (by setting $k = \log N$ in the above); this is discussed further in the Section ??.

By known constructions [40, 61] (see also [62]), the lower bound in Theorem ?? is tight up to $\log N$ factors and the factors depending only on w provided t is not too large. Hence, for all small enough even values of t and small enough w , we essentially completely resolve the cell probe complexity of representing k -wise independent distributions. The almost-matching upper bounds are nonconstructive, relying on unbalanced bipartite graphs with very strong unique-expansion properties; they yield data structures which are nonadaptive and have word length $w = 1$, so the tightness of Theorem ?? holds even in this restricted setting of nonadaptive bit probe data structures.

Our lower bound for k -wise independent problems is actually a bit stronger than stated above, as we are able to establish the same bound for problems which only exhibit γ -almost k -wise independence, for a nontrivially large value of γ . The stronger form (which is the form which will appear as Theorem ?? in Section ??) is as follows:

Theorem (Theorem ??, Strengthened To Almost Independence). *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a γ -almost k -wise independent distribution supported on its columns. Then the same lower bound as in the first presentation of Theorem ?? (above) holds for F provided $\gamma \leq N^t 2^{-O(ktw)}$.*

An important consequence of this improvement is that, using known constructions of almost independent distributions [63], in the case $t, w = O(1)$ we can set $k = O(\log N)$, $\gamma = N^{-O(1)}$ and obtain an explicit problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M \leq \text{poly}(N)$ and satisfying the lower bound $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$. This improvement will be necessary later on for obtaining polynomial time range avoidance algorithms with minimum possible stretch.

We next establish a lower bound that improves on the previous theorems in the special setting of *nonadaptive bit probe* data structures; remember that “bit probe” denotes the word length setting $w = 1$. For this lower bound, the hardness property of our data structure problems is *small bias* [63]: a distribution over $\{0, 1\}^N$ is γ -biased if it fools all parity functions with error $\leq \frac{\gamma}{2}$.

Theorem 1.21. *Let $t > 1$, $t = O(1)$ be a fixed odd number. Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a N^{-c} -biased distribution supported on its columns, for $c = O(1)$ a sufficiently large constant (depending on t). Then any nonadaptive bit probe data structure for F with space S and time t must satisfy:*

$$S \geq \tilde{\Omega}\left(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}}\right) \geq N^{\frac{2}{t} + \epsilon_t}$$

where $\epsilon_t > 0$ is a positive constant depending only on t .

Using explicit constructions of small bias probability spaces (again [63]), we obtain explicit problems $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$ which satisfy the lower bound $S \geq N^{\frac{2}{t} + \epsilon_t}$ for every odd

constant t , where $\epsilon_t > 0$ depends only on t . Small bias probability spaces were first used as a hard data structure problem in [41] to obtain near-maximal space lower bounds for nonadaptive bit probe data structures making $t = 2$ queries. We emphasize that, because of the optimality of the bound in Theorem ?? (there is an upper bound almost matching the bound in Theorem ?? even in the nonadaptive bit probe setting and even for odd values of t), the above Theorem ?? is provably unattainable using *only limited independence*. More formally, to construct any data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ satisfying a lower bound $S \geq N^{\frac{2}{t} + \epsilon}$ for any constants $t > 1$, $\epsilon > 0$, and $M \leq \exp(N^{o(1)})$, we provably cannot use k -wise independence as our only hardness property for the problem F . This result is therefore of special interest, as it the first example of a cell probe lower bound that is provably stronger than the best bound provable using limited independence alone: all of the previous best lower bound methods [40, 59, 60, 41, 42]⁹ can be carried out using limited independence as the sole hardness condition.

NC⁰ Range Avoidance Algorithms: In Section ?? we give the best known unconditional algorithms for NC⁰ range avoidance. Using our primary observation connecting range avoidance to data structure lower bounds together with known constructions of limited independence and small bias spaces [63], we can immediately obtain state of the art P^{NP} algorithms for NC⁰ range avoidance using our new cell probe lower bounds as a black box. In the case of Theorem ?? we get:

Theorem 1.22. *Let $t = O(1)$ be a fixed odd number. There is a polynomial time algorithm which outputs a list of strings in $\{0, 1\}^m$, such that for any NC_t⁰ generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with*

$$m \geq \tilde{O}(n^{\frac{t}{2} - \frac{t-2}{2(t+2)}})$$

one of the strings in the list must be a range-avoidance solution for G . In particular, there is a polynomial time NP-oracle algorithm for range avoidance on instances of the above form.

For our lower bound in Theorem ??, we are able to analyse it more effectively (rather than

⁹Viola's lower bound for $t = 2$ nonadaptive bit probe data structures [41] does not use k -wise independence, but k -wise independence was shown to achieve the same quantitative lower bound in the follow up work of [42].

applying it as a black box) and remove the use of an NP oracle in the associated range avoidance algorithm:

Theorem 1.23. *Let t, n be given with t even. There is a $m^{O(t^2)}$ -time algorithm which solves range avoidance given any NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with*

$$m \geq n^{\frac{t}{2}} \log n \cdot t^{\frac{t}{2} + o(t)}$$

More generally, the algorithm works if each output of G is computed by a depth t decision tree.

We emphasize the last point; not only do we decrease the required stretch quadratically for NC_t^0 functions compared to previous results [54, 55], but our algorithm also works in the more general setting that every output of G is a depth t decision tree over the inputs. The full statement of Theorem ?? is more general in two other ways which we didn't highlight above. First, it solves the harder *remote point* problem for G : for any constant $\epsilon > 0$, our algorithm will output a string which is $(\frac{1}{2} - \epsilon)$ -far in relative hamming distance from every string in the range of G . Second, it works if G is of the form $G : \Sigma^n \rightarrow \{0, 1\}^m$ for some nonboolean alphabet $|\Sigma| > 2$, and each output is computable by an adaptive decision tree querying at most t input symbols.

Small Bias Generators in NC^0 : In [50] it was shown that, if $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is an NC_t^0 generator such that $G(\mathbf{x})$ is $\text{poly}(n)^{-1}$ biased when $\mathbf{x}$ is uniformly distributed on $\{0, 1\}^n$, then we must have $m \leq O(2^t n^{\lceil \frac{t}{2} \rceil})$; in other words, once $m \gg n^{\lceil \frac{t}{2} \rceil}$ there is an $\mathbb{F}_2$ -linear test that can distinguish $G(\mathbf{x})$ from random. Our Theorem ?? improves this result for all odd values of t ; in particular, as an immediate corollary we obtain:

Corollary 1.24. *For every odd constant t there is $\epsilon_t > 0$, so that every NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is distinguished with $\text{poly}(n)^{-1}$ advantage by an $\mathbb{F}_2$ -linear test once $m \geq n^{\frac{t}{2} - \epsilon_t}$.*

We note that Theorem ?? in fact implies something much stronger, that $G(\mathbf{x})$ can be distinguished from uniform by linear tests for *any* distribution of $\mathbf{x} \in \{0, 1\}^n$. The original result in [50]

is specially tailored to the case where the random “seed” x is uniformly distributed on $\{0, 1\}^n$, and hence cannot be used to give a lower bound in the bit probe model. An interesting feature of this is the following: because $\mathbb{F}_2$ -linear tests can be performed by polynomial-size circuits, our result implies that every NC_t^0 PRG can be distinguished efficiently by *nonuniform* algorithms in the stretch regime $n \mapsto n^{\frac{t}{2}-\epsilon_t}$. However, in the typical approach to NC^0 PRG constructions pioneered by Goldreich [43], the generator G is sampled at random from a distribution of large ($\approx m$) entropy; in particular, such a cryptographic primitive does not consist of a single NC^0 PRG, but rather a *sampleable ensemble* of NC^0 PRGs. Since our Corollary ?? gives no indication as to how to *find* the linear distinguisher for G given the description of G , we seemingly cannot use it to attack Goldreich’s generator in the stretch regime $n \mapsto n^{\frac{t}{2}-\epsilon_t}$, even if we allow nonuniformity in our attacker.

Complexity-Theoretic Barriers to Higher Cell Probe Lower Bounds: Next, in Section ?? we use our main correspondence between cell probe lower bounds and local generators to establish *barrier results* for improving state of the art cell probe lower bounds. Our focus here is on the bit probe model (word length $w = 1$), and on nonadaptive data structures with time complexity $t = O(1)$. We show the following two barrier-type results:

Theorem 1.25 (Consequences of Explicit Data Structure Lower Bounds).

1. *If there is an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M \leq \exp(N^{o(1)})$ which requires space $S \geq N - N^{o(1)}$ for data structures making 4 nonadaptive bit probes, then $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$.*
2. *If there is a universal constant $\epsilon > 0$ and an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M = \text{poly}(N)$ requiring space $S \geq N^\epsilon$ for nonadaptive data structures making $O(1)$ bit probes, then there is a polynomial time NP oracle algorithm for NC^0 range avoidance with polynomial stretch.*

The conclusion in (1) above would be a monumental breakthrough in complexity theory and is considered extremely hard. The conclusion in (2) above was directly posed in [53] as an important

direction in unconditional range avoidance algorithms. We note that (1) of Theorem ?? has a strong similarity to results in [41] and [64]. [41] shows that improved data static data structure lower bounds imply breakthrough circuit lower bounds for multi-output functions. The implied circuit lower bounds are of two kinds: lower bounds against low-depth circuits over a complete basis of large fan-in gates, and lower bounds against linear size log-depth circuits with bounded fan-in gates. On the other hand, [64] shows that improved data structure lower bounds (of a different kind) imply breakthrough constructions of rigid matrices; these results are specific to *linear data structures* rather than general data structures as considered in this work and in [41].

Next, we turn toward a different and more novel kind of barrier in the realm of data structure lower bounds, based on the *natural proofs* paradigm of Razborov and Rudich [56]. Following Razborov-Rudich, we define a *natural data structure lower bound* to be some efficiently testable property of functions $F : [N] \times [M] \rightarrow \{0, 1\}$, so that random functions have the property, and any function with the property satisfies a corresponding cell probe lower bound (see Section ?? for a formal definition). We argue in Section ?? that the strongest lower bound methods in the cell probe model known prior to this work can be made *natural* in this sense. This continues to hold for our new lower bound in Theorem ?. On the other hand, we show that under two widely believed cryptographic assumptions, certain lower bounds in the bit probe model are unachievable using proofs of this nature:

Theorem 1.26 (Natural Proofs Barriers for Data Structure Lower Bounds).

1. Assume the existence of NC^1 PRGs. Then there is no natural proof of a space lower bound $S \geq N - N^{0.99}$ for data structures making 4 nonadaptive bit probes solving a problem $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$.
2. Assume existence of polynomial stretch NC^0 PRGs. Then for every $\epsilon > 0$ there is $t \in \mathbb{N}$ so that no natural proof can establish a lower bound $S \geq N^\epsilon$ for data structures making t nonadaptive bit probes solving a problem $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$.

As mentioned above our lower bound in Theorem ?? can be made natural, as well as all lower

bounds proven prior using the cell sampling method or communication arguments. However, interestingly, we *do not* know if the lower bound for small bias distributions in Theorem ?? can. It is therefore unclear at the moment whether or not the “natural properties barrier” for data structures, according to the definitions used above, has already been broken by the new results in this paper. Regardless, we argue in Section ?? that the lower bound in Theorem ?? is still “natural” in a weaker and less formal sense, and that significantly stronger lower bounds will probably have to differ from it in this regard assuming the nonuniform security of NC^0 PRGs. We discuss further the question of the “naturalness” of Theorem ?? in Section ??.

Approaches to Stronger Lower Bounds via Communication Complexity: Finally, in Section ?? we discuss possible approaches to proving lower bounds in the nonadaptive bit probe model of the form $S \geq N^\epsilon$ for time $t = O(1)$, where $\epsilon > 0$ is a fixed universal constant not depending on t ; this corresponds precisely to the regime in which the existence of cryptographically secure polynomial-stretch NC^0 PRGs yields a natural proofs barrier. We formulate cell probe lower bounds in this regime in terms of an equivalent communication model, reminiscent of PIR schemes, in which a single party Alice holding an input $x \in X$ communicates with a council of t Bobs holding a shared input $y \in Y$. We discuss an approach to analyzing this model based on “lifting” a query complexity measure investigated by [65, 66], and relate this model to other *high end* communication complexity classes such as PH^{cc} .

1.5.2 Subsequent Developments:

After the posting of an initial preprint of this work, two improvements to Theorem ?? were communicated to us by Weiqiang Yuan [67]:

1. Our lower bound $S \geq \tilde{\Omega}(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}})$ for small bias problems can be extended to hold for *adaptive* bit probe data structures.
2. In the case of *nonadaptive* bit probe data structures, the lower bound for small bias problems can be strengthened quantitatively to $S \geq \tilde{\Omega}(N^{\frac{2}{t-1}})$.

In both cases, it is required as in Theorem ?? that $t = O(1)$ is an odd constant.

1.5.3 Our Techniques: Derandomizing Semirandom CSP Refutations

We now discuss at a high level our approach to proving our main cell probe lower bound for k -wise independent problems (Theorem ??). We will narrow down our discussion in three respects: (1) we focus on the case $t = O(1)$ and ignore multiplicative factors depending on t in our space lower bound, (2) we restrict attention to *nonadaptive bit probe* data structures, and (3) we focus on achieving a lower bound of the form $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$ for k -wise independent problems in the special case $k = O(\log N)$.

In this setting, using our main observation connecting NC^0 generators and data structures, we need to show that any $O(\log N)$ -wise independent random variable $f^{\text{pseud}} \in \{0, 1\}^N$ has the following property: for any NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ with $S = \tilde{o}(N^{\frac{2}{t}})$, with nonzero probability $f^{\text{pseud}} \notin \text{range}(G)$. Say for a moment that we view a candidate generator G with the above parameters as a *cryptographic PRG*. In this case, a body of work dedicated to the cryptanalysis of NC^0 PRGs has shown that any such G can be distinguished by polynomial time algorithms; more strongly, there is a polynomial time algorithm which can generate certificates of the fact $f^{\text{unif}} \notin \text{range}(G)$ with very high probability when $f^{\text{unif}} \in \{0, 1\}^N$ is a uniformly random vector¹⁰. Our goal is to look at the certificates generated by these algorithms, and argue that they will still be produced with high probability when we replace the truly random vector f^{unif} with the *pseudorandom* vector f^{pseud} .

The task of certifying $f^{\text{unif}} \notin \text{range}(G)$ for a worst case NC_t^0 generator G is a special case of *semirandom CSP refutation*. In particular, given an NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ and any $f \in \{0, 1\}^N$, we may define a t -CSP instance with S variables and N constraints, which is satisfiable if and only if $f \in \text{range}(G)$: for each $x \in [N]$ with G_x depending on input cells

¹⁰Some NC^0 cryptanalysis techniques, e.g. Theorem 6 in [50], do not yield certificates $f \notin \text{range}(G)$, and only give statistical distinguishers which rely crucially on the fact that the *seed* of G is generated uniformly; such techniques seem to have little relevance to cell probe lower bounds.

$i_x^1, \dots, i_x^t \in [S]$, we add the constraint

$$G_x(E(i_x^1), \dots, E(i_x^t)) = f_x$$

where $E(1), \dots, E(S)$ are boolean-valued variables. If G is fixed and f^{unif} is uniformly random, we end up with a random distribution of t -CSP instances, where the *left hand side* of each constraint is fixed (worst-case), while the *right hand side* is sampled uniformly at random (average case). If we can give an efficient *refutation algorithm* which, with high probability over this distribution, outputs a certificate that the given CSP is *unsatisfiable*, then we can use it to distinguish the generator G : for a uniformly random right hand side the refutation algorithm will typically output “UNSAT,” while for $f \in \text{range}(G)$ the algorithm can never output “UNSAT.”

This hybrid worst-case/average-case model is known as the *semirandom CSP* model¹¹. This model was first introduced in [68] and has been the subject of many followups including [68, 51, 52, 69] (see the theses of Witmer and Manohar [70, 71] for a more in depth literature review). State of the art semirandom refutation algorithms ([52, 69, 72]) can certify $f^{\text{unif}} \notin \text{range}(G)$ for an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ whenever $m \geq \tilde{\omega}(n^{t/2})$. Taking $m = N$, $n = S$ this is precisely the parameter regime $S \leq \tilde{o}(N^{\frac{2}{t}})$ we aim to investigate in our cell probe lower bound for $O(\log N)$ -wise independent problems. It is considerably easier to carry out the arguments in [52, 69, 72] in the case t is even; we will only prove lower bounds for even values of t in this work and hence avoid the technical difficulties that arise for odd t .

In the case t is even, the refutation procedures in [52, 69, 72] can be divided into roughly three steps:

1. Generate from G a second NC_t^0 generator $G' : \{0, 1\}^{O(S)} \rightarrow \{0, 1\}^N$ which is $\mathbb{F}_2$ -linear (each output of G' XORs together t inputs); reduce the task of certifying $f \notin \text{range}(G)$ for an arbitrary NC_t^0 PRG G , to certifying that f is $(\frac{1}{2} - \epsilon)$ far in Hamming distance from

¹¹The semirandom CSP model is more general than the “random right hand side” variant introduced here, see e.g. [52] for a general definition and comparison to the special case presented here.

$\text{range}(G')^{12}$ for some small constant $\epsilon > 0$. This is referred to as *strong XOR refutation*.

2. Generate from G' a sequence of real matrices $A_1, \dots, A_N \in \mathbb{R}^{m \times m}$ for some $m = \text{poly}(N)$; reduce the task of certifying that f is far from $\text{range}(G')$ (strong XOR refutation) to certifying that the $\infty \rightarrow 1$ norm of the matrix

$$A[f] = \sum_{x \in [N]} (2f_x - 1)A_x$$

is small. The reader unfamiliar with the $\infty \rightarrow 1$ norm should think of it for now as close relative of the spectral norm which can be approximated in polynomial time (a formal definition is in Section ??).

3. Show that the $\infty \rightarrow 1$ norm of $A[f^{\text{unif}}]$ is small with high probability where f^{unif} is uniformly distributed on $\{0, 1\}^N$.

To carry out our lower bound, we need to modify the final point, and prove that it continues to hold with high probability when we replace the truly random f^{unif} with our pseudorandom f^{pseud} . This is argued in Section ?? and consists of two steps. The first is a reweighting trick from [69], which replaces the $A_1, \dots, A_N$ with a second sequence $B_1, \dots, B_N$ so that for any $f \in \{0, 1\}^N$, whenever $A[f]$ has a large $\infty \rightarrow 1$ norm, $B[f]$ has a large spectral norm ($B[f]$ is defined analogously to $A[f]$). We then need to prove that $B[f^{\text{pseud}}]$ has small spectral norm with high probability. For this we use the fact that, for any matrix D in m dimensions and any even number k , $\|D\|$ is approximated by the trace-moment polynomial $\text{tr}((D^\top D)^{k/2})^{1/k}$ within a factor $m^{1/k}$. Since in our case $B[f]$ lies in $\mathbb{R}^{m \times m}$ with $m \leq \text{poly}(N)$ and $k = O(\log N)$, we have that $\text{tr}((B[f]^\top B[f])^{k/2})^{1/k}$ approximates the spectral norm of $B[f]$ to within a constant factor. Since $\text{tr}((B[\cdot]^\top B[\cdot])^{k/2})^{1/k}$ is (the k^{th} root of) a degree k polynomial, we can argue that its distribution with respect to f^{pseud} and f^{unif} will be similar which will complete the proof.

There are several technical difficulties that we are leaving out of our discussion. First, to make our lower bound work in the setting of a general locally computable generator G (with larger

¹²We actually need both f and $\neg f$ to be far in Hamming distance from the range.

word length and adaptive decision trees at its outputs) we need a more involved argument in place of (1) above. Second, we need to exercise a bit more care in our analysis of the value of the trace moment polynomial $\text{tr}((B[f]^\top B[f])^{k/2})^{1/k}$ to achieve the same bounds in the case that our distribution on f is only $N^{-O(1)}$ -wise $O(\log N)$ -wise independent, rather than perfectly $O(\log N)$ -wise independent. Finally, to deal with the case $k \gg \log N$ we need to generalize (2) above, using another construction from semirandom CSP refutation called the “Kikuchi Matrix” [73, 69].

We will not sketch in any detail our proof of Theorem ?? which gives a lower bound $N^{\frac{t}{2} + \Omega(1)}$ for odd constants t in the nonadaptive bit probe model, however we note that it follows the same overall form: we take a semirandom refutation procedure (in this case, the refutations in Section 9 of [69] based on earlier work of [74]), and show that we can still find the associated certificates in the pseudorandom case, for some appropriate notion of pseudorandomness. In this case we rely instead on small bias (fooling parity functions) as our pseudorandom property in place of k -wise independence. As mentioned earlier in the introduction, it is provably necessary to move beyond k -wise independence to achieve a lower bound of this form.

1.5.4 Open Problems

TODO: update w venkat paper We present here three potentially tractable open problems we believe are of particular interest. We start with the (seemingly) most accessible problem:

Problem 1.27. *Extend the bound in Theorem ?? to hold for odd values of t , and the bound in Theorem ?? to hold for even values of t .*

The natural approach here is to utilize the CSP refutation procedures for odd order CSPs developed in [52, 69] and analyze them in the “pseudorandom case” as we have done for even order CSPs. It seems likely that a solution to the first part of Problem ?? would automatically solve the second part (see the discussion at the end of Section ??).

Our second problem relates to Theorem ?. While our lower bounds for k -wise independent data structure problems are known to be optimal up to lower order terms (for even t), our nonadaptive bit probe lower bounds for ϵ -biased distributions have no matching upper bound. We highlight

closing this gap as an important open problem:

Problem 1.28. *For $t \in \mathbb{N}$, determine the largest constant $\delta_t > 0$ so that, in the nonadaptive bit probe model, we have the lower bound $S \geq N^{\delta_t - o(1)}$ for any data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ which supports an $\exp(-N^{o(1)})$ -biased distribution on its columns.*

Our Theorem ?? yields the lower bound $\delta_t \geq \left(\frac{t}{2} - \frac{t-2}{2(t+2)}\right)^{-1}$, in particular $\delta_t > \frac{2}{t}$, for every odd value $t \geq 3$. Theorem ?? (the second form described above) also implies $\delta_t \geq \frac{2}{t}$ for even values of t since ϵ -almost k -wise independence is a special case of ϵ -biasedness; this latter lower bound holds in the more general adaptive setting and with larger word lengths. Both of these lower bounds hold even if we only aim to achieve bias $\text{poly}(N)^{-1}$ (rather than $\exp(-N^{o(1)})$ as stated above). On the other hand a construction of [50] implies that $\lim_{t \rightarrow \infty} \delta_t = 0$, in particular $\delta_t \leq O(\sqrt{\frac{1}{t}})$. Closing this gap in either direction remains an intriguing open problem.

Finally, we highlight two closely related problems: removing the NP oracle in Theorem ?? and removing the nonuniformity in Corollary ??:

Problem 1.29. *For NC_t^0 generators $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$:*

- (a) *for any $t \geq 3, \epsilon > 0$, give a subexponential time deterministic algorithm that solves range avoidance for G whenever $m \geq n^{\frac{t}{2} - \epsilon}$*
- (b) *for any $t \geq 5, \epsilon > 0$, give a subexponential time uniform algorithm that distinguishes G with subexponential advantage whenever $m \geq n^{\frac{t}{2} - \epsilon}$*

One approach to this problem is to prove an *algorithmically constructive* variant of the hypergraph Moore bound of [69, 72]. Such an algorithm would also improve the best known polynomial time refutation algorithm for random t -CSPs, and has been an open problem (in the case $t = 3$) since the original work of [74]; there appears to be some significant possibility that no such algorithm exists. A different approach, tailored specifically to (b), is to show that the lower bound in Theorem ?? can be made *natural* in the sense of Razborov-Rudich. More specifically, it would actually be sufficient to develop an efficient algorithm with the following behavior: for some absolute constant $c \in \mathbb{N}$, the algorithm takes as input a uniformly random set $A \subseteq \mathbb{F}_2^n$, $|A| = n^c$, and

with nonnegligible probability must output a certificate that A is a $\frac{1}{4}$ -biased probability space. See Section ?? for more details.

1.5.5 Prior Work on Data Structure Lower Bounds

We give here a slightly more involved historical account of lower bound techniques that can prove the strongest lower bounds in the static cell probe model for some explicit problem¹³. As mentioned earlier, the best known lower bound for general static data structures (for any $t > 1$ and any w) prior to our work was $S \geq N^{\frac{1}{t}}(\log M)^{1-\frac{1}{t}}$, first achieved by [40] for k -wise independent problems ($M \approx N^k$). Lower bounds of the form $S \gtrsim N^{\frac{1}{t}}$ can be established by a direct reduction to deterministic two party communication complexity. A more precise communication complexity analysis (keeping track of the number of rounds and the number of bits sent by each party in each round) developed in [75, 76] gives a similar but more precise lower bound in various settings. To achieve a lower bound which includes the extra factor $\approx \log M$ requires techniques more specially adapted to the cell probe model. This can be achieved either by expansion-based arguments known as *cell sampling* ([40, 61, 59, 60]), or in some cases by a more involved communication complexity argument using direct sum theorems [77].

In the special setting of *nonadaptive bit probe* data structures (the setting in which our Theorem ?? applies), two works have achieved lower bounds improving on the cell sampling bound. In [41], when the time is $t = 2$ a near-maximal space lower bound $S \geq \frac{N}{2}$ is shown for a problem with $M \leq N^{O(1)}$ datapoints. The lower bound applies to any $F : [N] \times [M] \rightarrow \{0, 1\}$ whose columns support a γ -biased distribution for γ a sufficiently small constant. In subsequent work of [42], a lower bound of the form $S \gtrsim N^{\frac{1}{t-1}}(\log M)^{1-\frac{1}{t-1}}$ is obtained for k -wise independent problems ($M \approx N^k$). Their method is a deterministic variant of cell sampling, and they prove that the given lower bound is essentially the best that cell-sampling type methods can achieve. [42] also improves quantitatively Viola’s bound for $t = 2$, showing a lower bound $S \geq (1 - \epsilon)N$ for any $N^{-O(1)}$ -almost $O(\log N)$ -wise independent problem where $\epsilon > 0$ is any constant (and the $O(\cdot)$

¹³A great deal of work in the field is dedicated to adapting these techniques to prove the best possible lower bounds for particular *natural* problems of interest, which we will not cover here.

terms hide linear dependencies on ϵ). The results in [42] can be applied to larger word lengths $w > 1$ provided we transition to data structure problems with nonboolean output, of the form $F : [N] \times [M] \rightarrow \{0, 1\}^w$.

Observe that, in the setting where M is not much larger than N , e.g. $M \leq \exp(\text{poly log } N)$, the factor $(\log M)^{1-\frac{1}{t}}$ has negligible effect on the cell sampling bound $S \geq N^{\frac{1}{t}} (\log M)^{1-\frac{1}{t}}$. In this setting, no previously known method (even cell sampling) gives a lower bound significantly stronger than the direct communication reduction, which yields $S \gtrsim N^{\frac{1}{t}}$. We explain this reduction here since it is so simple. Say F has a time t space S data structure with word length w . Consider a protocol in which Alice holds x , Bob holds y ; Bob constructs the data structure encoding $\mathcal{E}(y)$, Alice simulates the query procedure on x , requesting cells of $\mathcal{E}(y)$ from Bob who responds with their contents. Alice sends $\log S$ bits in each round to request a cell, Bob sends w bits to reveal the cell's contents, and communication lasts t rounds, so the overall communication is $t \log S + tw$. For any problem F with maximal two party communication complexity, e.g. $F : [N] \times [N] \rightarrow \{0, 1\}$ given by $F(x, y) = \mathbb{1}\{x = y\}$, we then have $t \log S + tw \geq \log N$, i.e. $S \geq N^{\frac{1}{t}} \cdot 2^{-w}$.

Finally it is important to keep in mind that for *non-explicit* problems, counting arguments show that most functions $F : [N] \times [N] \rightarrow \{0, 1\}$ require space very close to N or time very close to $\log N$ [78]. The situation here is similar to many other areas in complexity; a nonconstructive counting argument implies that random functions exhibit an extremely high lower bound, and the heart of the subject is to prove similar bounds for explicit (and ideally natural) problems.

1.6 How to Construct Random Strings

As in Chapter ??, our main result in this chapter is a deterministic algorithm for solving a wide class of explicit construction problems, whose correctness hinges on a certain computational hardness assumption. In contrast to Chapter ??, which gives algorithms for explicit construction problems reducible to Lossy Code, in this chapter our algorithms apply more generally to explicit construction problems that reduce to Avoid. This increased generality will come at a cost: (1) we will only be able to handle reductions that produce sufficiently *stretching* instances of Avoid,

and (2) the hardness assumptions we require here are *significantly* stronger than those required in Chapter ??.

Recall that in Chapter **TODO: ref chapter**, we identified a wide range of important unsolved explicit construction problems which all have a common complexity upper bound (they are reducible to Avoid). The primary motivating question behind the work in this chapter is the following:

Question 1: Is there compelling complexity-theoretic evidence that the explicit construction problems considered in Chapter **TODO: ref**, such as the construction of Ramsey graphs or rigid matrices, should be solvable in deterministic polynomial time?

Now, as per the results in Chapter **TODO: ref**, one way to obtain a positive answer to Question 1 would be to find a plausible complexity-theoretic assumption implying that Avoid is in FP. Unfortunately, **TODO: as mentioned previously?** it was shown in [79] that under plausible cryptographic and complexity-theoretic assumptions, Range Avoidance is *intractable* in general, so this general approach to Question 1 seems ill-founded. However the intractability of Range Avoidance in the worst case doesn't give evidence for the intractability of the explicit construction problems mentioned above - it could be that a *weaker* assumption than solving Range Avoidance is sufficient for efficient explicit constructions. Indeed such an assumption was identified in [79] - solving Range Avoidance for *uniformly generated* circuits. In particular, recall that an explicit construction is defined by some property $\Pi \subseteq \{0, 1\}^*$, and the goal is to produce a string in $\Pi \cap \{0, 1\}^n$ given 1^n as input. If the explicit construction problem defined by Π reduces in polynomial time to Avoid, then there is some *uniform sequence* of Avoid instances $C_n : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^{m(n)}$, such that C_n can be produced in $\text{poly}(n)$ time given 1^n ; any algorithm which can solve Avoid *only on this uniformly generated sequence of instances* would then suffice to produce explicit strings in Π . The tractability of Range Avoidance on uniform circuits is raised explicitly in [79]. It is also stated there that “some of the authors are skeptical that this special case of Avoid is easy.”

Aside from the application to explicit constructions and range avoidance, there is a second

self-contained motivation for the results of this chapter in terms of algorithmic information theory. Time-bounded Kolmogorov complexity K^T [80] is a fundamental measure of complexity of a Boolean string. Given a string x and a time bound T , $K^T(x)$ is the size of the smallest program p such that $\mathcal{U}(p)$ outputs x within T steps, where $\mathcal{U}$ is a universal Turing machine fixed in advance. Intuitively, for feasible time bounds $T(n)$, $K^{T(n)}(x)$ measures the inherent compressibility or “structure” in x from the point of view of efficient algorithms, in the sense that any x with low $K^{T(n)}$ complexity has a short description from which it can be recovered efficiently. Thus a string with high K^T complexity can be considered unstructured or random-like.

Given functions $T, \ell : \mathbb{N} \rightarrow \mathbb{N}$ and a sequence $(x_n)_{n \in \mathbb{N}}$ of strings, where each x_n is of length n , let us call the sequence (K^T, ℓ) -hard if $K^{T(n)}(x_n) \geq \ell(n)$ for all n . We are interested in the complexity of producing a (K^T, ℓ) -hard sequence of strings, for polynomial time bounds T and for ℓ as large as possible. Three particularly important settings we focus on here will be $\ell(n) = n^{\Omega(1)}$, $\ell(n) = \tilde{\Omega}(n)$, and $\ell(n) = n - o(n)$. A simple argument shows that K^T -hard strings cannot be produced in time $T' = o(T/\log(T))$ by an algorithm A which outputs x_n when given n in unary. Indeed, the existence of such an algorithm would imply that $K^T(x_n) = O(\log(n))$, as the universal machine $\mathcal{U}$ can output x_n in less than time T when given n in binary and a constant-size program for $\mathcal{A}$ ¹⁴. But is it possible for such a sequence to be produced by an algorithm running in time $\text{poly}(T)$?

Question 2: Let $T : \mathbb{N} \rightarrow \mathbb{N}$ be any time-constructible function and let $\ell(n) \in \{n^{\Omega(1)}, \tilde{\Omega}(n), n - o(n)\}$. Is there a deterministic algorithm $\mathcal{A}$, which given input 1^n , runs in time $\text{poly}(T)$ and outputs a (K^T, ℓ) -hard string?

Question 2 is a fundamental question about Kolmogorov complexity, for which we currently do not have clear positive or negative complexity-theoretic evidence. A positive answer to Question 2 would be very interesting, as it would demonstrate the power of having polynomially more time:

¹⁴Here we use the standard fact that any algorithm $\mathcal{A}$ running in time T' can be simulated by $\mathcal{U}$ in time $O(T' \log(T'))$.

a $\text{poly}(T)$ time algorithm would be capable of producing highly T -incompressible strings, while a $o(T/\log(T))$ time algorithm can only produce strings that are highly T -compressible. Indeed Hypothesis 1.19 in a recent work of [81] asks a very similar question¹⁵ to Question 2, and states that “The plausibility of Hypothesis 1.19 remains to be investigated”. Question 2 also has implications for the theory of meta-complexity, which has seen much recent work [82, 83, 84].

As the reader may have guessed by now, Questions 1 and 2 are in fact closely related; in particular, it was shown in [53] that a positive answer to Question 2 for polynomial time bounds T would imply a positive answer to Question 1 (this connection explained in detail in **TODO: later part**). We prefer to work with the Question 2 formulation for most of this chapter. A natural approach to Question 2 is via pseudo-randomness. Under the assumption that E requires non-deterministic circuits of size $2^{\epsilon n}$ for some constant $\epsilon > 0$, it is known [85, 86] that for any constant $c > 0$ there is a pseudo-random generator G_n with seed length $O(\log(n))$ and error $o(1)$ against co-non-deterministic circuits of size n^c , computable in time $\text{poly}(n)$. Consider $T(n) = n^d$, where $d < c$ is any constant. The property of being a K^T -incompressible string, i.e., a length- n string of K^T -complexity $\geq n - 1$, can be checked by a co-non-deterministic circuit of size at most n^c , and moreover at least half of all strings of length n satisfy the property. Hence close to half of the outputs of G_n satisfy the property of K^T -incompressibility, which means that the range of G_n is a $\text{poly}(n)$ size set S_n computable in time $\text{poly}(n)$ and containing at least one K^T -incompressible string for large enough n . However, it is unclear how to identify efficiently *which* of the strings in S_n is incompressible - this seems to require calls to an NP oracle. We could try and *combine* the strings in S_n together into a string that is K^t -hard, for example by concatenating them. This approach has two drawbacks:

1. Concatenating the strings yields a new string of length $\gg T(n)$ rather than length n , since the set S_n has size $\gg n^c$. Hence for any reasonable setting of $\ell(n)$, even $\ell(n) = n^{\Omega(1)}$, we do not get any interesting consequences for K^T -hardness when we are interested in time bounds as a *function* of input length.

¹⁵The difference is that instead of only asking about producing K^T -hard strings for time-constructible functions T , [81] asks for an algorithm that gets T and n as input in unary, and outputs a K^T -hard string of length n .

2. Even if we don't mind destroying the relation between the time bound and input length, concatenating $\text{poly}(n)$ strings which are maximally incompressible can, at best, achieve incompressibility $\ell(n) = n^{\Omega(1)}$. If we are in the more stringent compressibility settings $\ell(n) = \tilde{\Omega}(n)$ or $\ell(n) = n - o(n)$ which occur more frequently in explicit construction applications, concatenating $\text{poly}(n)$ many strings appears to be useless.

Like most of the interesting problems in derandomization, answering Question 2 *unconditionally* would be difficult, as any $(K^T, n^{\Omega(1)})$ -hard string for $T > n^2$ can be transformed easily into the truth table of a hard Boolean function in E, i.e., a function with circuit complexity at least $2^{\epsilon m}$ for some ϵ , and hence would imply $E \not\subseteq P/\text{poly}$. Instead, we would simply like complexity-theoretic evidence in favor of or against Question 2. Preferably, this evidence should avoid exotic assumptions, and involve standard beliefs about the difficulty of simulating one resource efficiently by another, e.g., the belief that time cannot be simulated by much smaller space or by much smaller non-uniformity.

1.6.1 Our Results

We give positive answers to Questions 1 and 2, under believable complexity assumptions. Our hardness assumptions are of the following two forms, each involving a parameter $d \in \mathbb{N}$:

Hypothesis (Hardness Assumptions for d^{th} -Exponential Time Bounds).

1. (Strong Form) *There is an absolute constant $\epsilon > 0$ so that the following holds. For any time constructible $T(n), m(n)$, with $T(n) \geq n$ having at most d^{th} -order exponential growth rate and $n \leq m(n) \leq \text{poly}(n)$, there is a function $f : \mathbb{N} \mapsto \{0, 1\}^*$, $|f(n)| = m(n)$ computable in $\text{poly}(T(n))$ time, which cannot be computed in time $T(n)^\epsilon$ with $m(n)^\epsilon$ bits of advice by any NP-oracle machine for more than finitely many n .*
2. (Weak Form) *Let $T(n) = \exp^{[d+1]}(n)$. There is some $\epsilon > 0$ and a language computable in time $T(n)$ which is not computable in time $T(n)^\epsilon$ with an NP oracle and $2^{\epsilon n}$ bits of advice even infinitely often.*

We are using $\exp^{[d]}(n)$ for the d -fold iteration of the exponential function $\exp(n) = 2^n$. Such assumptions, particularly of the second kind, are fairly standard in complexity theory. Indeed, when $d = 0$, the second assumption is a standard derandomization assumption, asserting that there is a language in $\text{TIME}[2^n]$ which does not have NP oracle circuits of size $2^{o(n)}$. The main novelty in our case is that we use these assumptions for larger values T , in particular for time bounds T which have iterated-exponential growth rate. While we state the strong form assumption by quantifying over all time constructible $T(n), m(n)$, in fact we only require the assumption to hold for quite a limited class of bounds: the bounds $T(n)$ needed are representable by constant length arithmetic expressions with $\exp$, $\log$, and ceiling/floor functions, while the bounds $m(n)$ needed are simply of the form $m(n) = \exp(c \lceil \log n \rceil)$ for constants c .

The first assumption is analogous to assumptions made in the recent literature on hardness vs randomness [87], and generalizes the second assumption in a natural way. Both the first and second assumptions involve natural beliefs about the relationships between the fundamental resources of time, non-determinism and advice. Both assumptions formalize the intuition that time cannot be sped up by an arbitrary polynomial amount using non-determinism and advice. Indeed, note that there is no known way to speed up time even by a *super-constant* amount by using non-determinism and advice.

We believe that our assumptions hold relative to a *random oracle*, and here is an informal argument for the case of the second assumption. Given oracle A , define the time T bounded Turing machine M^A to accept x of length n if the x 'th string of length $T(n)$ is in A (where we consider the lexicographic order on strings). For a random oracle A , the truth table of A at length $T(n)$ is Kolmogorov-random even conditioned on truth tables below length $T(n)$. Since $T(n)^\epsilon$ time non-deterministic oracle machines can only access strings of A at length $T(n)^\epsilon$ or below, it should not be possible to compute the first 2^n bits of A at length $T(n)$ from this information together with arbitrary $T(n)^\epsilon$ bits of advice.

Our approach to using these assumptions in service of constructing random strings is via pseudo-randomness, and is a variation on the strategy we critiqued a few paragraphs ago: con-

struct a pseudorandom generator, enumerate its range, and concatenate the constituent strings. The problems with this approach discussed in the last section boiled down to one issue: for a standard complexity-theoretic PRG fooling polynomial size nondeterministic circuits, we can at best achieve a seed length of $O(\log n)$, and hence will end up needing to concatenate a very long list of $\text{poly}(n)$ strings. To remedy the situation, it would suffice to use pseudorandom generators with much smaller seed length of $o(\log n)$ or even $\log \log n$. While it is known that $O(\log n)$ seed length is information-theoretically optimal for fooling *nonuniform* circuits of size $\text{poly}(n)$, we aim to exploit the uniformity of the adversary we are fooling to get seed length *as small as possible*; for uniform adversaries the lower bound of $\log n$ no longer applies. This yields a result which is interesting in its own right: efficient constructions of small pseudo-random sets for uniform (or even slightly non-uniform) algorithms, under assumptions of the form discussed in the previous paragraph.

Theorem 1.30 (Informal, see Theorems ?? and ??). *Under the Hypotheses above with $d = 1$, there is a pseudorandom generator with seed length $O(\log \log n)$ which fools $\text{TIME}^{\text{NP}}[\text{poly}(n)]$; if we assume the strong form the generator runs in polynomial time, and if we assume the weak form it runs in quasipolynomial time.*

More generally, under the above hardness assumption with large parameters d (i.e. for higher-exponential time bounds), we obtain pseudorandom generators fooling $\text{TIME}^{\text{NP}}[\text{poly}(n)]$ with seed length $O(\log^{[d+1]}(n))$ (for infinitely many n). If we assume the strong form hypothesis our generators will run in polynomial time, and if we assume the weak form they will run in iterated quasipolynomial time. When $d > 1$, our construction will require in addition that the input length n is of the form $n = \exp^{[d-1]}(n')$ for some n' , or more generally that $K^{\text{poly}(n)}(n) \leq \log^{[d]}(n)$.

Some terms in the above require clarification. We are using $\log^{[d]}(\cdot)$ to denote the d -fold iteration of the logarithmic function. Our notion of “iterated quasipolynomial time” will be defined in the preliminaries - it describes a family of runtime bounds which grows faster than polynomial and quasipolynomial, but is much closer to polynomial than to exponential. Finally, the notation $K^{\text{poly}(n)}(n)$ denotes the time bounded Kolmogorov complexity of the *number* n , when written as a

string in standard binary notation.

Our PRG construction can actually handle advice (nonuniformity) up to $\log^{[d]}(n)$ when the seed length is $\log^{[d+1]}(n)$, which we observe is optimal via a standard argument (Lemma ??). It also holds for fooling larger time bounds $\text{TIME}^{\text{NP}}[T(n)]$ for $T(n) \gg \text{poly}(n)$ (at the cost of the time complexity of the generator being low as a function of $T(n)$ rather than of n), and for other oracles $\mathcal{O}$ in place of the NP oracle (provided we use $\mathcal{O}$ in place of NP in the hardness assumption).

We then turn our attention briefly to PRGs fooling uniform algorithms *without* NP oracles: what is the minimal seed length such that we can fool $\text{TIME}[T(n)]$ by a PRG with running time $T(n)^{O(1)}$? For this problem we show in Theorem ?? that the machinery developed above is unnecessary: if the PRG is capable of simulating the distinguishers it is trying to fool, there is a straightforward method which can reduce seed length $O(\log n)$ to arbitrarily small seed length. Hence under the standard assumptions that achieve logarithmic seed length for $\text{TIME}[T(n)]/n$ (namely that E requires exponential circuit size) we can obtain an arbitrarily strong reduction in seed length for $\text{TIME}[T(n)]$. Like our NP-oracle result, this result applies to distinguishers with mild nonuniformity, and the achievable seed length scales with the uniformity in a way that is provably optimal according to Lemma ??.

Random Strings and Explicit Constructions: We next use our short-seed pseudorandom generators for P^{NP} (Theorem ??) to give (conditional) constructions of K^{poly} -random strings. Our first construction of random strings is the following, which achieves incompressibility $\tilde{\Omega}(n)$:

Theorem 1.31 (Informal, see Theorems ?? and ??). *Under our main hardness assumption with $d = 1$, for any $k \in \mathbb{N}$ there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(1^n)$ outputs an n -bit string x satisfying $\text{K}^{n^k}(x) \geq \frac{n}{\text{polylog}(n)}$ for all n ; the algorithm will run in polynomial time if we use the strong form of the assumption, and quasipolynomial time if we use the weak form.*

More generally under our main hardness assumption for larger values of d , we obtain constructions of strings with $\text{K}^{n^k}(x) \geq \frac{n}{\text{poly}(\log^{[d]}(n))}$. Using the strong form of the assumption the construction runs in polynomial time, and under the weak form it runs in iterated quasipolynomial

time. When $d > 1$ we require the integer n to be of the form $n = \exp^{[d-1]}(n')$ for some integer n' , or more generally to satisfy $K^{\text{poly}(n)}(n) \leq \log^{[d]} n$.

If we are satisfied with our construction algorithms succeeding on some unknown infinite set of input lengths, we are able to bootstrap the above constructions so that the $\log^{[d]}(n)$ loss in K -complexity becomes additive rather than multiplicative:

Theorem 1.32 (Informal, see Theorem ??). *Assume the first version of our main hardness assumption for all d . Then for any $k, d \in \mathbb{N}$ there is a polynomial time algorithm $\mathcal{A}$ such that $\mathcal{A}(1^n)$ outputs an n -bit string x satisfying $K^{n^k}(x) \geq n - \log^{[d]} n$ for infinitely many n .*

In Section ?? we then use the previous results to give conditional polynomial time explicit constructions of various important combinatorial objects shown reducible to Range Avoidance in [88]. We will defer most of the specifics to Section ??, but highlight the following two results:

Theorem (See Theorems ?? and ??).

1. *Under the strong form of our main hardness assumption with $d = 2$, there is a $\text{poly}(n)$ time algorithm which produces a matrix $M \in \mathbb{F}_2^{n \times n}$ which is Valiant-Rigid whenever n is a power of 2.*
2. *Assuming that there is a language in $\text{TIME}[2^{2^n}]$ that is not computable in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ (even infinitely often)¹⁶, there is a language in EXP that requires Boolean circuits of size $\frac{2^n}{\text{poly}(n)}$ for all n .*

Crucially, our results give the first standard-form hardness assumptions under which the above explicit construction problems (and others in Section ??) have an efficient algorithm, and they do so via a universal approach: we have a single object, K^{poly} -random strings, whose efficient explicit construction follows from plausible hardness assumptions, and which automatically satisfies various other pseudorandom properties, e.g. rigidity as a matrix or maximal hardness as a Boolean function.

¹⁶This is a particular case of the weak form hardness assumption.

Further Applications: In Sections ?? and ?? we discuss two further applications of our random string constructions. The first shows a “hardness condensation” phenomenon for the main hardness assumptions we use in this paper (with the lower bound relativized to a PSPACE oracle), whereby we may amplify the degree of nonuniformity in the lower bounds from 2^{n^ϵ} to $2^{n-o(n)}$. The second addresses the question of the relative difficulty of *uniform* range avoidance and general (nonuniform) range avoidance posed in [81, 79], where we observe using our main results and some previous hardness results ([79, 89]) that under plausible hardness assumptions, nonuniform range avoidance is significantly harder than the uniform variant.

Barriers to Improving Our Main Results: In the final section we give two sets of results indicating barriers to improving our main PRGs and random string constructions. Our first result casts our iterated-logarithmic seedlength PRG fooling P^{NP} (Theorem ??) as a reconstructive *multi-source* extractor, similar to Trevisan’s analysis [90] of the classical hardness randomness connection in [91, 85] in terms of reconstructive single-source extractors. We show that any such extractor must have iterated-logarithmic seed length, and thus that any improvement to the seed length of our PRG in Theorem ?? must use substantially different techniques.

Second, we give an oracle separation showing that it is not possible via black-box arguments to obtain our main results from more standard assumptions like $E \not\subseteq \text{SIZE}[2^{\epsilon n}]$. In [88] it is shown that explicit construction of $(K^{\text{poly}}, n - 1)$ random strings and $2^{\epsilon n}$ hard truth tables are equivalent with respect to NP oracle reductions. If such an equivalence held without NP oracles it would supersede our main results here: we could start with an assumption that E requires $2^{\epsilon n}$ -size circuits, and then use the reduction to produce from the hard truth table a string with high time bounded Kolmogorov complexity. We show that no such reduction exists which is *relativizing*. This justifies in some sense the need for more high-end hardness assumptions to achieve our main results.

1.6.2 Overview of Main Construction

We describe at a high level our construction of pseudorandom generator with small seed length secure against any $L \subseteq \{0, 1\}^n$ computable in $\text{TIME}^{\text{NP}}[\text{poly}(n)]$. Using the classical hardness-randomness connection, under the assumption that $\text{TIME}[2^{O(n)}]$ requires exponential size NP-oracle circuits we can obtain a generator $G^0 : \{0, 1\}^{s_0(n)} \rightarrow \{0, 1\}^n$ with seed length $s_0(n) \leq O(\log n)$. The key observation is that this first attempt at a generator significantly overshoots our primary goal in one respect: the generator G^0 obtained from the generic hardness-randomness connection would in fact be secure against $\text{TIME}^{\text{NP}}[\text{poly}(n)]/n$ machines which have access to n bits of advice. On the other hand we only want a generator which fools *uniform* algorithms.

To use this to our advantage, we consider a recursive argument. Let $n_1 = s_0(n)$, and consider n_1 as our *new input length*. Define the language $L_1 \subseteq \{0, 1\}^{n_1}$ consisting of the seeds z of G^0 such that $G^0(z) \in L$. We know that $\Pr_{z \sim \{0, 1\}^{n_1}}[z \in L_1] \approx \Pr_{x \sim \{0, 1\}^n}[x \in L]$ by the security of our first generator G_0 , and therefore if we could find a second generator $G_1 : \{0, 1\}^{s_1(n_1)} \rightarrow \{0, 1\}^{n_1}$ which fools the language L_1 , we would find that the composition $G_0 \circ G_1$ fools the original language L . If $s_1(n_1) = O(\log n_1) = O(\log \log n)$ we would have made significant progress. For such a generator G^1 to fool L_1 , we would need it to fool NP-oracle algorithms with running time $\text{poly}(n)$; however the input length of L_1 is $n_1 = O(\log n)$, so as a function of the new input length we need G^1 to fool algorithms running in *exponential time* with an NP oracle. We may proportionally increase the allowable runtime of G^1 to exponential as well; so overall we have scaled the time complexities of our generator/distinguisher by an exponential. However, crucially, the *advice complexity* of deciding L_1 remains $O(1)$: this is where we use crucially the uniformity of our distinguisher. If L required n bits of advice compute, then the best advice upper bound we could place on L_1 is $n \approx 2^{n_1}$ which is trivial.

Continuing in this way, we are able to iteratively reduce the seed length of our original generator by a logarithmic composition, for an arbitrary constant number of phases. The cost is that we need to obtain, in each step, pseudorandom generators with logarithmic seed length that fool algorithms running in $\exp(\exp(\cdots \exp(n) \cdots))$ time with an NP oracle, and which are computable

in a comparable amount of time (without an NP oracle). To achieve such generators we rely on the standard toolkit of hardness-randomness transformations [91, 85, 92]. In this way, we can argue the security of our construction based on the kind of hardness assumptions described above.

There are a few additional intricacies that we are skimming over here, which contribute to the two caveats in our main theorem statements: the weaker runtime bounds when using the second version of our hardness assumptions, and the requirement that the number n is of the form $n = \exp^{[d-1]}(n')$ for some n' when using the first version of our hardness assumptions. The first of these issues roughly stems from the fact that, when trying to apply our hardness assumption to get an $O(\log n_j)$ seed length generator on input length $n_j \approx \log^{[j]}(n)$ (which we will then compose with the outer generator to reduce the total seed length by another logarithm), we will actually need to apply our hardness assumption on some other input length $m_j = O(n_j)$; this will mean that we can at best hope for our generator to run in time $\text{poly}(\exp^{[j]}(O(n_j))) \approx \exp^{[j]}(O(\log^{[j]} n))$, which will be superpolynomial when $j > 1$. This issue can be avoided by considering the more refined first version of our hardness hypothesis.

For the second issue, it turns out that we can only guarantee the security of our generator on inputs lengths n such that the *number* n itself has time-bounded Kolmogorov complexity $\log^{[d]}(n)$. For $d = 1$ this changes nothing, since every number n has $K^{O(n)}(n) \leq \log n$ via its binary representation, so we achieve a generator with seed length $O(\log \log n)$ which is valid on all input lengths, but for large values of d our construction is only valid for infinitely many n (in particular it will work for all n of the form $\exp^{[d-1]}(n')$ for some n'). The reason for this issue is as follows: in the above exposition, we considered the language $L_1 \subseteq \{0, 1\}^{n_1}$ of seeds mapping to strings in the base language L , and argued it is computable in time exponential in its input length. When we continue this argument for more steps and obtain languages $L_2, L_3, \dots$, we would like to argue at each step that they are uniformly computable with a small amount of advice (comparable to the input length they are defined on). However, the definition of these languages depends on the original input length n that we started on, and since $\log^{[d]}(\cdot)$ is not injective for any $d > 0$, we cannot determine n solely from the smaller input lengths, but will need to somehow supply the number n

as advice. If we assume the number n is sufficiently compressible, we are able to skirt this issue and complete the argument.

1.6.3 Related Work

We have already mentioned connections of our work to meta-complexity and Range Avoidance. In terms of derandomizing uniform algorithms, there has been a lot of work on uniform hardness-randomness tradeoffs, beginning with [93], but much of that work is not relevant to our setting as the emphasis is on uniform assumptions for derandomization, while we are interested instead in decreasing the seed length, and are not concerned with uniformity of the assumption.

However the recent work of [87] on super-fast derandomization is indeed very relevant from a technical point of view. They give a (conditional) derandomization of $\text{BPTIME}[T(n)]$ into $\text{TIME}[T(n)^{1+\epsilon}n]$. The classical approach to derandomizing $\text{BPTIME}[T(n)]$ would be to model it as a circuit of size $T(n)$ with input length $T(n)$ (the input represents the randomness) and use a PRG fooling this class. To fool this class of circuits requires a PRG with seed length $\log T(n)$ and running time $T(n)$, and if $T(n) = n^3$ we could therefore not hope to achieve derandomization in time $T(n)^{1+\epsilon}n$ no matter how fast the PRGs runtime is. A key observation in [87] is that that this approach to derandomizing $\text{BPTIME}[T(n)]$ actually *overshoots* what's required: if we model the $\text{BPTIME}[T(n)]$ machine more precisely as a $\text{TIME}[T(n)]$ machine on input length $T(n)$ with nonuniformity $n \ll T(n)$, then we only need a PRG which fools $\text{TIME}[T(n)]/n$; for this task, it is possible (at least information-theoretically) to achieve a seed length of $\log n$ or more generally $(1 + \epsilon) \log n$ rather than $\log T(n)$, which means that the enumeration of seeds will only cost us $n^{1+\epsilon}$ time.

They then proceed to construct a PRG for $\text{TIME}[T(n)]/n$ with seed length $(1 + \epsilon) \log n$ and computable in time $T(n)^{1+\epsilon}$. In their case a great deal of work must be dedicated to optimizing the runtime of the PRG which has no relevant parallel in our work; however the question of reducing the seed length down to the true level of uniformity, and the assumptions under which they are able to achieve it, have similarities to ours. In particular, they require assumptions of the form

$\text{TIME}[T(n)] \not\subseteq \text{TIME}[T(n)^{1-\delta}]/2^{(1-\delta)n}$ where $T(n) = 2^{kn}$ for a large constant k .

The innovation in our work is the use of recursion to achieve dramatically smaller seed length against inefficient uniform adversaries using a small amount of non-uniformity, which enables us to address Questions 1 and 2.

Chapter 1: References

- [1] L. Chen, S. Hirahara, and H. Ren, “Symmetric exponential time requires near-maximum circuit size,” in *56th Annual Symposium on Theory of Computing*, 2024.
- [2] Z. Li, “Symmetric exponential time requires near-maximum circuit size: Simplified, truly uniform,” in *56th Annual Symposium on Theory of Computing*, 2024.
- [3] R. M. Karp and R. J. Lipton, “Some connections between nonuniform and uniform complexity classes,” in *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*, ser. STOC ’80, Los Angeles, California, USA: Association for Computing Machinery, 1980, 302–309, ISBN: 0897910176.
- [4] R. Kannan, “Circuit-size lower bounds and non-reducibility to sparse sets,” *Information and Control*, vol. 55, no. 1, pp. 40–56, 1982.
- [5] R. Kleinberg, O. Korten, D. Mitropolsky, and C. Papadimitriou, “Total Functions in the Polynomial Hierarchy,” in *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*, J. R. Lee, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 185, Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2021, 44:1–44:18, ISBN: 978-3-95977-177-1.
- [6] P. Beame, S. Cook, J. Edmonds, R. Impagliazzo, and T. Pitassi, “The relative complexity of np search problems,” *Journal of Computer and System Sciences*, vol. 57, no. 1, pp. 3–19, 1998.
- [7] M. Chiari and J. Krajíček, “Witnessing functions in bounded arithmetic and search problems,” *Journal of Symbolic Logic*, vol. 63, no. 3, pp. 1095–1115, 1998.
- [8] N. Vyas and R. Williams, “On Oracles and Algorithmic Methods for Proving Lower Bounds,” in *14th Innovations in Theoretical Computer Science Conference (ITCS 2023)*, Y. Tautman Kalai, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 251, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 99:1–99:26, ISBN: 978-3-95977-263-1.
- [9] L. Fortnow and T. Yamakami, “Generic separations,” *Journal of Computer and System Sciences*, vol. 52, no. 1, pp. 191–197, 1996.
- [10] P. Beame, “A switching lemma primer,” *Technical Report, Department of Computer Science, University of Washington*, 1994.
- [11] A. Razborov, *Bounded Arithmetic and Lower Bounds in Boolean Complexity, In Feasible Mathematics II*. Birkhauser, 1993.
- [12] P. Beame, R. Impagliazzo, J. Krajíček, T. Pitassi, P. Pudlák, and A. R. Woods, “Exponential lower bounds for the pigeonhole principle,” in *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, S. R. Kosaraju, M. Fellows, A. Wigderson, and J. A. Ellis, Eds., ACM, 1992, pp. 200–220.
- [13] M. L. Furst, J. B. Saxe, and M. Sipser, “Parity, circuits, and the polynomial-time hierarchy,” *Math. Syst. Theory*, vol. 17, no. 1, pp. 13–27, 1984.

- [14] M. Ajtai, “ Σ^1_1 -formulae on finite structures,” *Ann. Pure Appl. Log.*, vol. 24, no. 1, pp. 1–48, 1983.
- [15] J. Paris, A. Wilkie, and A. R. Woods, “Provability of the pigeonhole principle and the existence of infinitely many primes,” *J. Symb. Log.*, vol. 53, pp. 1235–1244, 1988.
- [16] A. Russell and R. Sundaram, “Symmetric alternation captures bpp,” *computational complexity*, vol. 7, pp. 152–162, 1998.
- [17] R. Canetti, “More on bpp and the polynomial-time hierarchy,” *Information Processing Letters*, vol. 57, no. 5, pp. 237–241, 1996.
- [18] A. Maciel, T. Pitassi, and A. R. Woods, “A new proof of the weak pigeonhole principle,” *J. Comput. Syst. Sci.*, vol. 64, no. 4, pp. 843–872, 2002.
- [19] S. R. Buss, L. A. Kolodziejczyk, and N. Thapen, “Fragments of approximate counting,” *J. Symb. Log.*, vol. 79, no. 2, pp. 496–525, 2014.
- [20] A. Atserias and N. Thapen, “The ordering principle in a fragment of approximate counting,” *Electron. Colloquium Comput. Complex.*, vol. TR13-149, 2013. ECCC: TR13-149.
- [21] A. C. Yao, “Theory and application of trapdoor functions,” in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 80–91.
- [22] N. Nisan and A. Wigderson, “Hardness vs randomness,” *Journal of Computer and System Sciences*, vol. 49, no. 2, pp. 149–167, 1994.
- [23] R. Impagliazzo and A. Wigderson, “P = BPP if E requires exponential circuits: Derandomizing the XOR lemma,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC ’97, El Paso, Texas, USA: Association for Computing Machinery, 1997, 220–229, ISBN: 0897918886.
- [24] D. H. J. Polymath, “Deterministic methods to find primes,” *arXiv*, 2010.
- [25] I. C. Oliveira and R. Santhanam, “Pseudodeterministic constructions in subexponential time,” in *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2017, Montreal, Canada: Association for Computing Machinery, 2017, 665–677, ISBN: 9781450345286.
- [26] M. Agrawal, N. Kayal, and N. Saxena, “Primes is in P,” *Ann. of Math*, vol. 2, pp. 781–793, 2002.
- [27] V. Kabanets, “Easiness assumptions and hardness tests: Trading time for zero error,” *Journal of Computer and System Sciences*, vol. 63, no. 2, pp. 236–252, 2001.
- [28] R. Impagliazzo, V. Kabanets, and A. Wigderson, “In search of an easy witness: Exponential time vs. probabilistic polynomial time,” *Journal of Computer and System Sciences*, vol. 65, no. 4, pp. 672–694, 2002, Special Issue on Complexity 2001.
- [29] R. Williams, “Improving exhaustive search implies superpolynomial lower bounds,” *SIAM Journal on Computing*, vol. 42, no. 3, pp. 1218–1244, 2013. eprint: <https://doi.org/10.1137/10080703X>.

- [30] W. Maass, “Quadratic lower bounds for deterministic and nondeterministic one-tape turing machines,” in *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’84, New York, NY, USA: Association for Computing Machinery, 1984, 401–408, ISBN: 0897911334.
- [31] J. Robson, “Deterministic simulation of a single tape turing machine by a random access machine in sub-linear time,” *Information and Computation*, vol. 99, no. 1, pp. 109–121, 1992.
- [32] L. Fortnow, “Time–space tradeoffs for satisfiability,” *Journal of Computer and System Sciences*, vol. 60, no. 2, pp. 337–353, 2000.
- [33] L. Fortnow, R. Lipton, D. van Melkebeek, and A. Viglas, “Time-space lower bounds for satisfiability,” *J. ACM*, vol. 52, no. 6, 835–865, 2005.
- [34] R. Williams, “Inductive time-space lower bounds for sat and related problems,” *Computational Complexity*, vol. 15, pp. 433–470, 2005.
- [35] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology — CRYPTO ’87*, C. Pomerance, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1988, pp. 369–378, ISBN: 978-3-540-48184-3.
- [36] O. Goldreich, S. Goldwasser, and S. Micali, “How to construct random functions,” *J. ACM*, vol. 33, no. 4, 792–807, Aug. 1986.
- [37] E. Jeřábek, “Dual weak pigeonhole principle, boolean complexity, and derandomization,” *Annals of Pure and Applied Logic*, vol. 129, no. 1, pp. 1–37, 2004.
- [38] O. Korten, “The hardest explicit construction,” in *62nd Annual Symposium on Foundations of Computer Science*, 2021.
- [39] P. Elias and R. A. Flower, “The complexity of some simple retrieval problems,” *J. ACM*, vol. 22, no. 3, 367–379, Jul. 1975.
- [40] A. Siegel, “On universal classes of fast high performance hash functions, their time-space tradeoff, and their applications,” in *30th Annual Symposium on Foundations of Computer Science*, 1989, pp. 20–25.
- [41] E. Viola, “Lower bounds for data structures with space close to maximum imply circuit lower bounds,” *Theory of Computing*, vol. 15, no. 1, pp. 1–9, 2019.
- [42] A. Golovnev, T. Gur, and I. Shinkar, “Derandomization of cell sampling,” in *Symposium on Simplicity in Algorithms (SOSA)*, 2023, pp. 278–284.
- [43] O. Goldreich, *Candidate one-way functions based on expander graphs*, Cryptology ePrint Archive, Paper 2000/063, 2000.
- [44] M. Cryan and P. B. Miltersen, “On pseudorandom generators in nc_0 ,” in *Mathematical Foundations of Computer Science 2001: 26th International Symposium, MFCS 2001 Mariánské Lázně, Czech Republic, August 27–31, 2001 Proceedings 26*, Springer, 2001, pp. 272–284.
- [45] B. Applebaum, “Cryptographic hardness of random local functions,” *Comput. Complex.*, vol. 25, no. 3, 667–722, Sep. 2016.

- [46] B. Applebaum, Y. Ishai, and E. Kushilevitz, “Cryptography in nc^0 ,” *SIAM Journal on Computing*, vol. 36, no. 4, pp. 845–888, 2006.
- [47] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, “Cryptography with constant computational overhead,” in *Proceedings of the fortieth annual ACM symposium on Theory of computing*, 2008, pp. 433–442.
- [48] A. Jain, H. Lin, and A. Sahai, “Indistinguishability obfuscation from well-founded assumptions,” in *Proceedings of the 53rd annual ACM SIGACT symposium on theory of computing*, 2021, pp. 60–73.
- [49] A. Jain, H. Lin, and A. Sahai, “Indistinguishability obfuscation from l_{pn} over $f_{p, dlin}$, and prgs in nc^0 ,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2022, pp. 670–699.
- [50] E. Mossel, A. Shpilka, and L. Trevisan, “On epsilon-biased generators in nc^0 ,” in *Annual Symposium on Foundations of Computer Science*, Citeseer, vol. 44, 2003, pp. 136–145.
- [51] S. R. Allen, R. O’Donnell, and D. Witmer, “How to refute a random csp,” in *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, IEEE, 2015, pp. 689–708.
- [52] J. Abascal, V. Guruswami, and P. K. Kothari, “Strongly refuting all semi-random boolean csps,” in *Proceedings of the Thirty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA ’21, Virtual Event, Virginia: Society for Industrial and Applied Mathematics, 2021, 454–472, ISBN: 9781611976465.
- [53] H. Ren, R. Santhanam, and Z. Wang, “On the range avoidance problem for circuits,” in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 640–650.
- [54] V. Guruswami, X. Lyu, and X. Wang, “Range avoidance for low-depth circuits and connections to pseudorandomness,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2022)*, Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2022.
- [55] K. Gajulapalli, A. Golovnev, S. Nagargoje, and S. Saraogi, “Range Avoidance for Constant Depth Circuits: Hardness and Algorithms,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2023)*, N. Megow and A. Smith, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 275, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 65:1–65:18, ISBN: 978-3-95977-296-9.
- [56] A. A. Razborov and S. Rudich, “Natural proofs,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 24–35, 1997.
- [57] E. Viola, “The complexity of distributions,” *SIAM Journal on Computing*, vol. 41, no. 1, pp. 191–218, 2012. eprint: <https://doi.org/10.1137/100814998>.
- [58] M. Patrascu, “Succincter,” in *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, IEEE, 2008, pp. 305–313.
- [59] R. Panigrahy, K. Talwar, and U. Wieder, “Lower bounds on near neighbor search via metric expansion,” in *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, IEEE, 2010, pp. 805–814.

- [60] K. G. Larsen, “Higher cell probe lower bounds for evaluating polynomials,” in *Proceedings of the 2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, ser. FOCS ’12, USA: IEEE Computer Society, 2012, 293–301, ISBN: 9780769548746.
- [61] A. Siegel, “On universal classes of extremely random constant-time hash functions,” *SIAM J. Comput.*, vol. 33, no. 3, 505–543, Mar. 2004.
- [62] H. Lefmann, P. Pudlák, and P. Savicky, “On sparse parity check matrices,” *Designs, Codes and Cryptography*, vol. 12, no. 2, pp. 107–130, 1997.
- [63] J. Naor and M. Naor, “Small-bias probability spaces: Efficient constructions and applications,” in *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, ser. STOC ’90, Baltimore, Maryland, USA: Association for Computing Machinery, 1990, 213–223, ISBN: 0897913612.
- [64] Z. Dvir, A. Golovnev, and O. Weinstein, “Static data structure lower bounds imply rigidity,” in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2019, Phoenix, AZ, USA: Association for Computing Machinery, 2019, 967–978, ISBN: 9781450367059.
- [65] P. Hrubes and A. Rao, “Circuits with medium fan-in,” in *30th Conference on Computational Complexity (CCC 2015)*, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015, pp. 381–391.
- [66] V. Lecomte, P. Ramakrishnan, and L.-Y. Tan, “The composition complexity of majority,” *arXiv preprint arXiv:2205.02374*, 2022.
- [67] W. Yuan, “Personal communication,”
- [68] U. Feige, “Refuting smoothed 3cnf formulas,” in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS’07)*, IEEE, 2007, pp. 407–417.
- [69] V. Guruswami, P. K. Kothari, and P. Manohar, “Algorithms and certificates for boolean csp refutation: Smoothed is no harder than random,” in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2022, Rome, Italy: Association for Computing Machinery, 2022, 678–689, ISBN: 9781450392648.
- [70] D. Witmer, “Refutation of random constraint satisfaction problems using the sum of squares proof system,” Ph.D. dissertation, PhD thesis, Carnegie Mellon University, 2017.
- [71] P. Manohar, “New spectral techniques in algorithms, combinatorics, and coding theory: The kikuchi matrix method,” Ph.D. dissertation, Weizmann Institute, 2019.
- [72] J.-T. Hsieh, P. K. Kothari, and S. Mohanty, “A simple and sharper proof of the hypergraph moore bound,” in *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2023, pp. 2324–2344. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611977554.ch89>.
- [73] A. S. Wein, A. El Alaoui, and C. Moore, “The kikuchi hierarchy and tensor pca,” in *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2019, pp. 1446–1468.
- [74] U. Feige, J. H. Kim, and E. Ofek, “Witnesses for non-satisfiability of dense random 3cnf formulas,” in *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS’06)*, 2006, pp. 497–508.

- [75] P. B. Miltersen, “Lower bounds for union-split-find related problems on random access machines,” in *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*, ser. STOC ’94, Montreal, Quebec, Canada: Association for Computing Machinery, 1994, 625–634, ISBN: 0897916638.
- [76] P. B. Miltersen, N. Nisan, S. Safra, and A. Wigderson, “On data structures and asymmetric communication complexity,” *Journal of Computer and System Sciences*, vol. 57, no. 1, pp. 37–49, 1998.
- [77] M. Patrascu and M. Thorup, “Time-space trade-offs for predecessor search,” in *Symposium on the Theory of Computing*, 2006.
- [78] P. B. Miltersen, “The bit probe complexity measure revisited,” in *STACS 93: 10th Annual Symposium on Theoretical Aspects of Computer Science Würzburg, Germany, February 25–27, 1993 Proceedings 10*, Springer, 1993, pp. 662–671.
- [79] R. Ilango, J. Li, and R. R. Williams, “Indistinguishability obfuscation, range avoidance, and bounded arithmetic,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, B. Saha and R. A. Servedio, Eds., ACM, 2023, pp. 1076–1089.
- [80] M. Li and P. M. B. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications, 4th Edition* (Texts in Computer Science). Springer, 2019, ISBN: 978-3-030-11297-4.
- [81] H. Ren, R. Santhanam, and Z. Wang, “On the range avoidance problem for circuits,” in *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, IEEE, 2022, pp. 640–650.
- [82] E. Allender, “The complexity of complexity,” in *Computability and Complexity - Essays Dedicated to Rodney G. Downey on the Occasion of His 60th Birthday*, A. R. Day, M. R. Fellows, N. Greenberg, B. Khoussainov, A. G. Melnikov, and F. A. Rosamond, Eds., ser. Lecture Notes in Computer Science, vol. 10010, Springer, 2017, pp. 79–94.
- [83] S. Hirahara, “Meta-computational average-case complexity: A new paradigm toward excluding heuristica,” *Bull. EATCS*, vol. 136, 2022.
- [84] Z. Lu and I. C. Oliveira, “Theory and applications of probabilistic kolmogorov complexity,” *Bull. EATCS*, vol. 137, 2022.
- [85] R. Impagliazzo and A. Wigderson, “P = bpp if e requires exponential circuits: Derandomizing the xor lemma,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC ’97, El Paso, Texas, USA: Association for Computing Machinery, 1997, 220–229, ISBN: 0897918886.
- [86] A. R. Klivans and D. van Melkebeek, “Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses,” in *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*, ser. STOC ’99, Atlanta, Georgia, USA: Association for Computing Machinery, 1999, 659–667, ISBN: 1581130678.
- [87] L. Chen and R. Tell, “Simple and fast derandomization from very hard functions: Eliminating randomness at almost no cost,” in *STOC*, ACM, 2021, pp. 283–291.

- [88] O. Korten, “The hardest explicit construction,” in *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, IEEE, 2021, pp. 433–444.
- [89] S. Aaronson, H. Buhrman, and W. Kretschmer, “A qubit, a coin, and an advice string walk into a relational problem,” in *15th Innovations in Theoretical Computer Science Conference, ITCS 2024, January 30 to February 2, 2024, Berkeley, CA, USA*, V. Guruswami, Ed., ser. LIPIcs, vol. 287, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 1:1–1:24.
- [90] L. Trevisan, “Extractors and pseudorandom generators,” *J. ACM*, vol. 48, no. 4, 860–879, Jul. 2001.
- [91] N. Nisan and A. Wigderson, “Hardness vs randomness,” *Journal of Computer and System Sciences*, vol. 49, no. 2, pp. 149–167, 1994.
- [92] C. Umans, “Pseudo-random generators for all hardnesses,” in *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, ser. STOC ’02, Montreal, Quebec, Canada: Association for Computing Machinery, 2002, 627–634, ISBN: 1581134959.
- [93] R. Impagliazzo and A. Wigderson, “Randomness vs. time: De-randomization under a uniform assumption,” in *39th Annual Symposium on Foundations of Computer Science, FOCS ’98, November 8-11, 1998, Palo Alto, California, USA*, IEEE Computer Society, 1998, pp. 734–743.